

基于机器学习的分布式系统故障分类系统设计与实现

摘要：在分布式系统中，节点故障会沿拓扑结构传播，导致自身及邻接节点的 KPI 指标和日志出现异常，因此高效的故障检测对系统稳定至关重要。本研究针对中国软件杯设计大赛提供的分布式数据库故障数据集展开分析，该数据集包含 107 个 KPI 特征和 6 类故障标签（含无故障类别），数据存在以下几种问题：严重的数据缺失（所有样本均有缺失值，77 个特征存在缺失，平均缺失率 8.44%）并且数据的缺失模式属于多变量随机缺失，缺失机制为 NMAR 和类别不平衡（0 类样本占比超 50%）问题。

文章中提出了一套针对性的数据预处理策略：首先对数据集进行数去重操作将 10000 条训练样本精简为 6296 条有效数据，旨在减少重复信息对后续模型训练的干扰；通过使用多重插补 MCMC 法进行数据填充，与未填充数据集进行对比，发现数据不做任何处理效果较好。推断缺失值本身可作为故障传播的证据，所以选择保留缺失值而非填充；针对类别不平衡问题，利用 LightGBM 模型内置的样本权重机制提升少数类关注度；同时移除 3 个常数特征，以及进行特征工程，避免冗余信息干扰。

在模型构建与优化阶段，以 LightGBM 为基础模型，结合遗传算法、贝叶斯优化和网格搜索的混合策略进行超参数调优，有效提升模型对复杂数据的拟合能力。进一步引入 XGBoost 和 CatBoost 作为基模型，通过加权投票融合策略实现优势互补，最终构建的集成模型在验证集上的 Macro-F1 分数从初始 LightGBM 的 0.75 提升至 0.855，显著提升了分类水平。

关键词：分布式系统故障检测；数据缺失；类别不平衡；LightGBM；超参数混合优化策略；模型融合

Design and Implementation of a Fault Classification System for Distributed Systems Based on Machine Learning

Abstract:In distributed systems, node failures can propagate along the topology, causing anomalies in KPI metrics and logs of the affected node and its adjacent nodes. Therefore, efficient fault detection is crucial for system stability. This study analyzes the distributed database fault dataset provided by the China Software Cup Design Competition, which contains 107 KPI features and 6 types of fault labels (including the no-fault category). The dataset has several issues: severe data missing (all samples have missing values, 77 features have missing values, with an average missing rate of 8.44%) and the missing pattern is multivariate random missing, with the missing mechanism being NMAR and class imbalance (0-class samples account for over 50%).

This paper proposes a set of targeted data preprocessing strategies: Firstly, duplicate data is removed from the dataset, reducing 10,000 training samples to 6,296 valid data, aiming to reduce the interference of duplicate information on subsequent model training. By using the multiple imputation MCMC method for data filling and comparing with the filled dataset, it is found that no data processing yields better results. Missing values themselves can serve as evidence of fault propagation, so they are retained instead of being filled. For the class imbalance problem, the LightGBM model's built-in sample weight mechanism is utilized to increase the attention to the minority class. At the same time, 3 constant features are removed, and feature engineering is conducted to avoid redundant information interference.

In the model construction and optimization stage, LightGBM is used as the base model, and a hybrid strategy combining genetic algorithm, Bayesian optimization, and grid search is employed for hyperparameter tuning, effectively enhancing the model's fitting ability for complex data. Further, XGBoost and CatBoost are introduced as base models, and a weighted voting fusion strategy is used to achieve complementary advantages. The final constructed ensemble model's Macro-F1 score on the validation set is improved from the initial LightGBM's 0.75 to 0.855, significantly enhancing the classification performance.

Key words: Distributed System Fault Detection; Missing data; Class imbalance; LightGBM; Hyperparameter hybrid optimization strategy; Model fusion

目录

摘 要.....	i
Abstract.....	ii
第一章 绪论.....	1
1.1 研究背景和意义.....	1
1.2 国内外研究现状.....	2
1.2.1 传统故障检测方法.....	2
1.2.2 基于机器学习的故障检测方法.....	2
1.2.3 当前研究面临的挑战和趋势.....	3
1.3 小结.....	4
第二章 数据分析与预处理.....	5
2.1 数据背景及介绍.....	5
2.2 数据完整性分析.....	5
2.3 类别分布分析.....	9
2.4 特征分布分析.....	10
2.5 数据预处理.....	11
2.5.1 数据去重.....	12
2.5.2 数据缺失值处理.....	12
2.5.3 数据类别不平衡处理.....	13
2.5.4 数据规范化.....	14
2.5.5 特征工程.....	15
2.6 小结.....	17
第三章 模型训练及优化.....	19
3.1 LightGBM 模型.....	19
3.1.1 基于直方图的决策树构建.....	19
3.1.2 梯度单边采样.....	19
3.1.3 互斥特征捆绑.....	20
3.2 评价指标.....	20
3.3 实验设计.....	20
3.3.1 数据准备.....	20

3.3.2 模型训练	21
3.3.3 模型评估	21
3.4 结果展示	21
3.4.1 整体性能评估	21
3.4.2 类别间性能对比	21
3.5 参数调优	22
3.5.1 遗传算法	22
3.5.2 贝叶斯优化	24
3.5.3 网格搜索	26
3.5.4 结果分析	27
3.6 模型融合	27
3.7 小结	30
第四章 系统设计	32
4.1 需求分析	32
4.1.1 应用场景	32
4.1.2 系统角色	32
4.1.3 功能需求	32
4.1.4 性能需求	32
4.2 系统设计	32
4.2.1 系统总体功能设计	33
4.2.2 数据库设计	33
4.3 系统实现	37
4.3.1 开发环境与技术选型	37
4.3.2 核心模块实现	38
4.3.3 前后端交互	39
4.3.4 用户界面展示	39
4.4 小结	41
第五章 总结与展望	42
5.1 总结	42
5.2 未来展望	42
参考文献	44

谢 辞.....	46
----------	----

第一章 绪论

1.1 研究背景和意义

随着信息技术的飞速发展，分布式系统因其高可扩展性、高可用性和高容错性等优势，成为支撑现代信息社会运行的关键基础设施。分布式系统广泛应用于云计算、大数据处理、物联网、金融服务和电子商务等领域。但是由于分布式系统本身的复杂性也带来了许多问题。分布式系统的节点数量庞大、组件繁多、节点间交互错综复杂，任何一个节点的硬件故障、软件缺陷、网络波动或配置错误都可能引发故障。更加复杂的是，故障往往会沿着系统的拓扑结构传播，导致一系列连锁反应，影响邻接节点乃至整个系统的关键性能指标（Key Performance Indicators, KPIs）和日志记录出现异常，严重时甚至会导致系统的服务中断，造成难以估计的影响。

传统的系统运维方式依赖于运维工程师的经验，通过人工监控和分析大量的 KPI 数据和日志来判断系统状态以及排查故障。这种方式不仅效率低下、响应缓慢，而且在面对庞大和复杂的分布式系统时，其局限性愈发凸显。KPI 数据通常维度高、数量大，日志信息的相关数据体量巨大并且结构不一致，人工难以实时、准确地从中提取有效故障特征。因此，研究和开发高效、智能的故障检测系统，对于保障分布式系统的稳定运行、提升服务质量、降低运维成本具有至关重要的现实意义。

机器学习作为人工智能的重要内容，凭借其强大的数据分析和模式识别能力，为解决分布式系统故障检测问题提供了新的途径^[1]。通过学习历史运行数据中的正常行为特征和故障特征，机器学习模型能够自动、快速地识别系统异常，甚至预测潜在故障。本研究旨在针对分布式系统，设计并实现一个基于机器学习的故障检测系统。该系统致力于解决实际应用中常见的挑战，例如本研究所用数据集中存在的严重数据缺失问题和类别不平衡问题。通过对 KPI 数据的深度分析和有效处理，结合先进的机器学习算法和优化策略，期望能够构建一个高精度、高效率的故障检测模型，辅助运维工程师快速定位和诊断故障类型，从而提高分布式系统的可靠性和可维护性，为相关领域的技术发展和应用实践提供有价值的参考。

1.2 国内外研究现状

分布式系统故障检测一直是学术界和工业界关注的热点研究方向。随着系统规模和复杂度的不断提升，研究者们提出了不同的诊断方法，大体上可以分为传统方法和基于机器学习的方法两大类。

1.2.1 传统故障检测方法

传统故障检测方法主要分为以下三类。

基于阈值的方法：这是最简单直观的一类方法，通过为关键性能指标（KPI）设定静态或动态阈值，当指标值超出预设范围时则判断为异常。例如，当 CPU 利用率持续高于 90% 或网络延迟突然增大数倍时触发告警。此类方法简单易实现，但缺点也十分明显：阈值的设定高度依赖专家经验和具体系统环境，难以适应系统动态变化和多样化的故障模式，容易产生较高的误报率和漏报率^[2]。

基于统计模型的方法：这类方法利用统计学原理对系统行为进行建模。例如，时间序列分析模型（如 ARIMA、Holt-Winters）可以预测 KPI 的未来趋势，当实际值与预测值偏差过大时则认为是异常。此外，统计过程控制图（如 CUSUM、EWMA）也被用于监控系统指标的稳定性。这些方法对具有明显统计特性的数据较为有效，但对于高维、非线性和复杂依赖关系的分布式系统数据，其建模能力有限^[3]。

基于依赖关系的方法：通过分析系统组件之间的调用关系、通信模式等构建依赖图或因果图，当某个组件发生故障时，根据依赖关系推断可能受影响的其他组件。此类方法有助于理解故障传播路径，但构建准确的依赖模型本身就是一个挑战，尤其是在微服务等动态变化的架构中^[4]。

1.2.2 基于机器学习的故障检测方法

基于机器学习的故障检测方法近年来取得了显著进展，展现出更强的适应性和更高的准确率。

监督学习方法：当拥有充足且准确标注的故障数据时，监督学习算法如支持向量机、决策树、随机森林、梯度提升机（如本研究中重点应用的 LightGBM）以及深度神经网络等可以学习从 KPI 特征到故障类别的映射关系。国内外许多研究利用这些模型在特定数据集上取得了良好的分类效果。然而，在实际生产环境中，获取全面、均衡的故障标签数据成本高昂，且系统可能出现新的、未曾标记

过的故障类型，这是监督学习方法面临的主要挑战^[5]。

无监督学习方法：此类方法无需故障标签，通过学习正常数据的模式来识别与之不符的异常点。常用的算法包括聚类算法（如 K-Means、DBSCAN）、基于密度的方法（如 LOF）、主成分分析以及自编码器等深度学习模型。无监督方法能够发现未知类型的故障，但通常误报率相对较高，且难以直接判断异常的具体原因和类型，需要结合专家知识进行后续分析^[6]。

半监督学习方法：介于监督学习和无监督学习之间，利用少量已标记数据和大量未标记数据进行训练。这种方法试图结合两者的优点，在标记数据稀缺的情况下提升检测性能^[7]。

深度学习在故障检测中的应用：随着深度学习技术的发展，循环神经网络及其变体如长短期记忆网络、门控循环单元因其处理时序数据的能力，被广泛应用于 KPI 异常检测。卷积神经网络也被用于从日志数据中提取模式或将 KPI 数据转化为图像进行分析。近年来，Transformer 模型也开始在时间序列异常检测领域展现潜力。深度学习模型能够自动提取复杂特征，但通常需要大量的训练数据和计算资源，且模型的可解释性较差^[8]。

1.2.3 当前研究面临的挑战和趋势

数据质量问题：实际生产环境中的 KPI 数据往往存在缺失值、噪声、高维度和类别不平衡（故障样本远少于正常样本）等问题，严重影响模型性能。如何有效处理这些问题是当前研究的重点之一，例如本研究中对数据缺失和类别不平衡的处理策略^[9]。

系统复杂性与动态性：微服务架构、容器化技术使得分布式系统更加灵活但也更加复杂，组件间的依赖关系动态变化，故障模式也随之演变，要求故障检测模型具有良好的适应性和泛化能力。

实时性要求：故障检测系统需要在故障发生后尽快告警，以便及时止损，这对模型的计算效率提出了较高要求。

可解释性：仅仅检测出异常或判断出故障类别往往不够，运维人员更希望了解故障发生的原因和模型的判断依据。因此，提升机器学习模型（尤其是深度学习模型）的可解释性是重要的研究方向。

多模态数据融合：分布式系统的状态信息不仅体现在 KPI 数据中，还蕴含在日志、追踪数据、配置信息等多种数据源中。如何有效融合这些多模态数据以提

升故障检测的准确性和全面性是一个热门趋势。

根因分析与故障预测：更进一步的研究不仅关注故障检测，还致力于故障的根因定位和故障预测，从而实现更主动、更智能的运维。

1.3 小结

综上所述，机器学习在分布式系统故障检测方面的研究取得了巨大的进步，但是仍然面临一些问题，在后续的工作中要着重考虑模型鲁棒性的提高、模型的实时性、可解释性及针对更复杂的动态系统来发展更加智能的运维方案，最后逐步向多技术融合方向发展并研究整个系统的端到端智能运维方案。本文立足于此，在一定数据集的基础上，针对其中的问题进行有目的的探究与应用，希望为机器学习方法在分布式系统故障检测的应用提供一定的参考。

第二章 数据分析与预处理

在机器学习领域中往往流传着一句话“数据和特征决定了机器学习的上限，而模型和算法只是逼近这个上限而已”，由此可见一个数据集的质量高低，往往会对机器学习任务产生重大的影响。所以在本章节重点对该研究涉及的数据集进行数据质量分析，而在数据质量分析问题中，重点围绕以下两点进行分析：数据完整性分析和数据分布分析。

2.1 数据背景及介绍

在分布式系统中某个节点发生故障时，故障会沿着分布式系统的拓扑结构进行传播，造成自身节点及其邻接节点相关的 KPI 指标和发生大量日志异常。本次比赛提供分布式数据库的故障特征数据和标签数据，其中特征数据是系统发生故障时的 KPI 指标数据，KPI 指标包括由 feature0、feature1...feature106 共 107 个指标，标签数据为故障类别数据，共 6 个类别，用 0、1、2、3、4、5 分别表示 6 个故障，其中 0 类代表无故障，数据集预览如表 2.1 所示。

表 2.1 数据集预览

sample_id	feature0	feature1	feature2	feature3	...	feature106	label
0	51.56724	288358.4		1.85589985	...	180.9773100	4
1	63.80487	288358.4	1.106802	1.05038663	...	314.7832630	5
2	49.13852	288358.4	1.1116490	0.76712683	...	183.3046102	2
3		288358.4	1.1091685		...	212.3026697	2
4	76.52083	288358.4	1.1134102	5.79540833	...	135.5412331	2
5	...	...	...	...	...	...	...

本研究涉及的数据均来源于中国软件杯设计大赛，提供了训练集和验证集，其中训练集的样本数据量为 10000 条，验证集的样本数据量为 1000 条。

2.2 数据完整性分析

数据完整性，主要指的是数据集是否存在数据缺失项。在实际项目中，数据缺失是一个普遍且复杂的问题，尤其对依赖数据驱动的智能系统（如 AI、机器人、工业互联网等）而言，数据缺失可能导致系统性能下降、决策偏差甚至失效。本研究的数据集存在着严重的数据缺失问题（如图 2.1 所示）。

为了更加详细地了解数据缺失情况，从四个维度切入，分别是样本缺失值情况、特征缺失值情况、数据缺失模式和数据缺失机制。

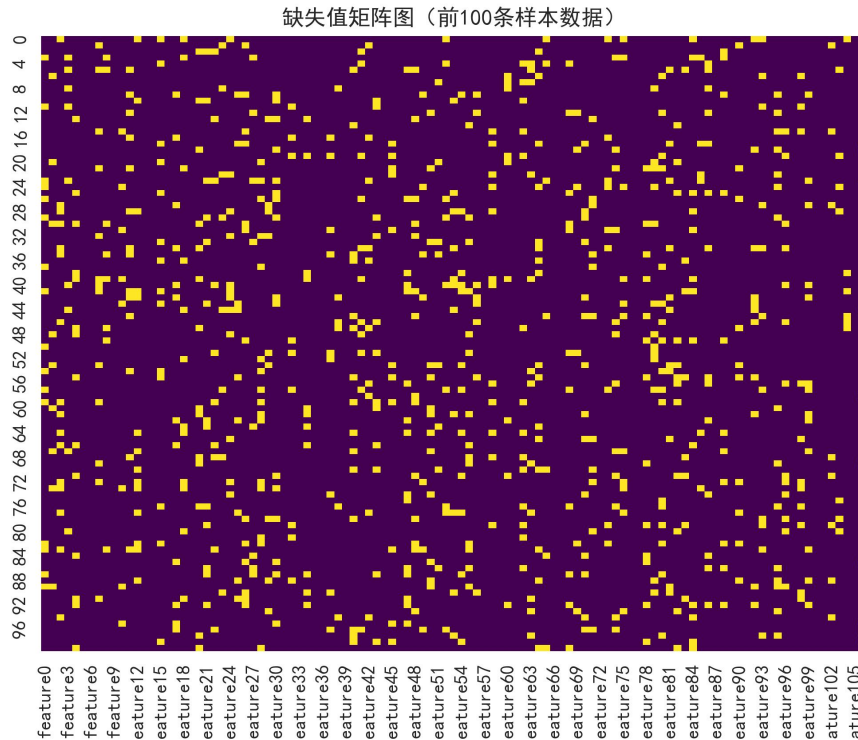


图 2.1 数据缺失值热力图（前 100 行数据）

样本缺失情况：在该部分针对每个样本缺失情况进行统计。在 10000 条样本数据中，含有缺失值的样本数量为 10000 条，也就是说所有样本均含有缺失值，不含有缺失值的样本数量为 0，并且大部分样本都存在 8-12 个缺失值，少量样本缺失值数量超 5 过 20。这意味着，在样本层面，给定的样本数据集，都存在数据完整性问题。样本缺失值情况如图 2.2 所示。

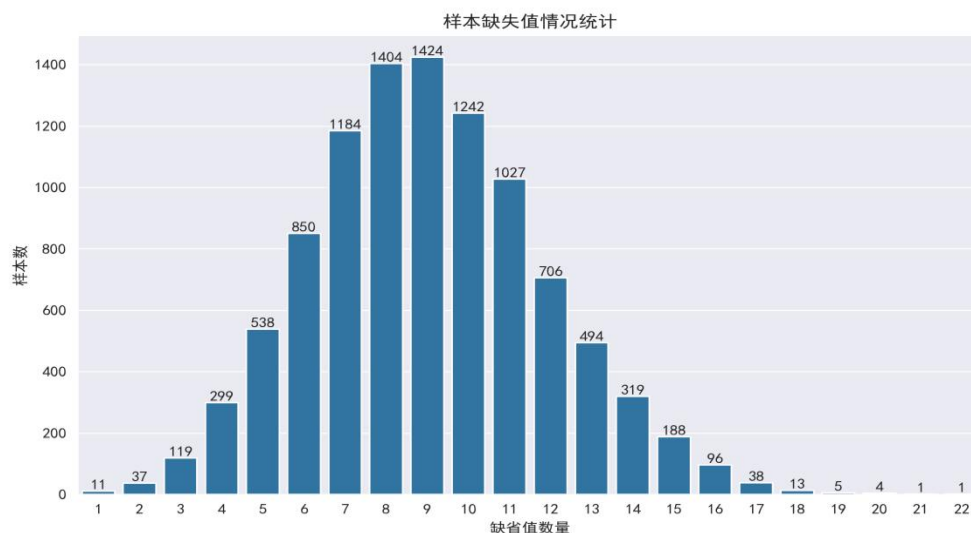


图 2.2 样本缺失值情况统计

特征缺失情况：针对数据集，对 feature0、feature1...feature106 共 107

个特征的缺失值数量做了相应的统计。从统计结果发现，有 77 个特征存在着缺失值，30 个特征没有缺失值，覆盖率约 72%。进一步分析发现，缺失值特征的评价缺失率在 8.44%。虽然缺失值覆盖了大部分特征，但每个特征的缺失比例相对较低。这可能意味着缺失值的处理不会对整体数据分布造成过大的影响。对于存在缺失值的特征，需要结合特征的重要性评估，判断是否需要保留或删除这些特征。如果某些特征的重要性较低且缺失率较高，可以考虑直接删除。各特征缺失情况如图 2.3 所示。

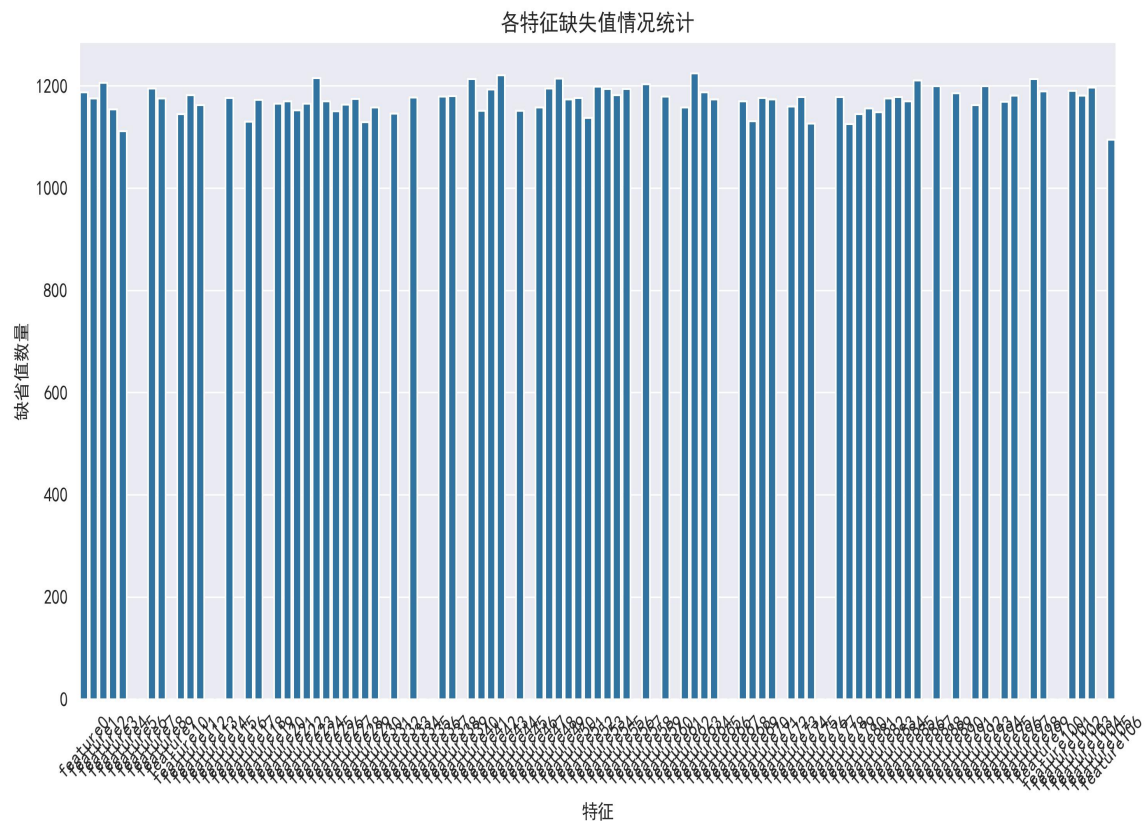
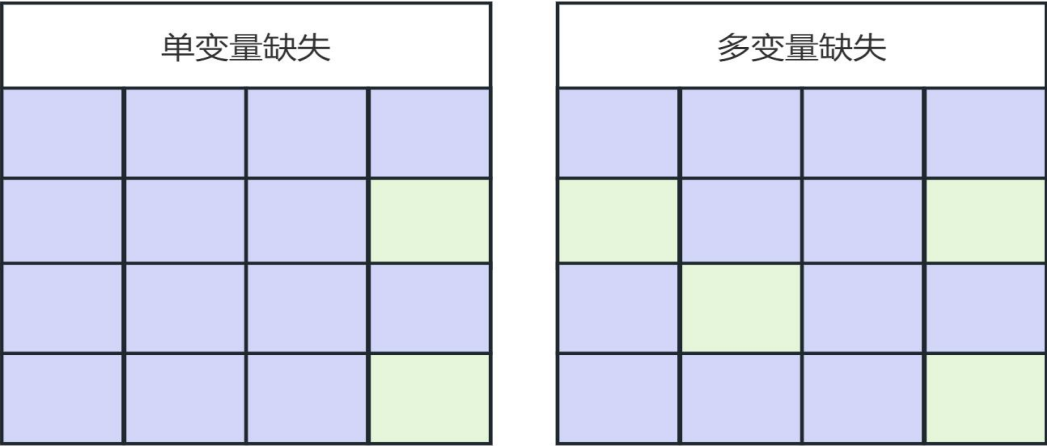


图 2.3 各特征缺失值情况统计

数据缺失模式：是指在数据集中，观测值与缺失值分布的结构化形式。它描述了缺失值在不同变量和观测对象中的分布特征，是数据分析中评估数据质量的重要概念。关于数据缺失模式以及后续会提到的数据缺失机制相关概念，是由学者 Rubin 在 1975 年发表的论文《INFERENCE AND MISSING DATA》^[10]一文中提及，而后续研究者（如 Enders, 2010）进一步完善了缺失模式的分类和应用场景。数据缺失模式分类主要有两个方向，一个是按照数据包含缺失变量个数来分类，一个是按照数据缺失值的结构来分类^[11]。

在按照缺失值变量个数分类这个角度来说，主要分为以下各类，单变量缺失

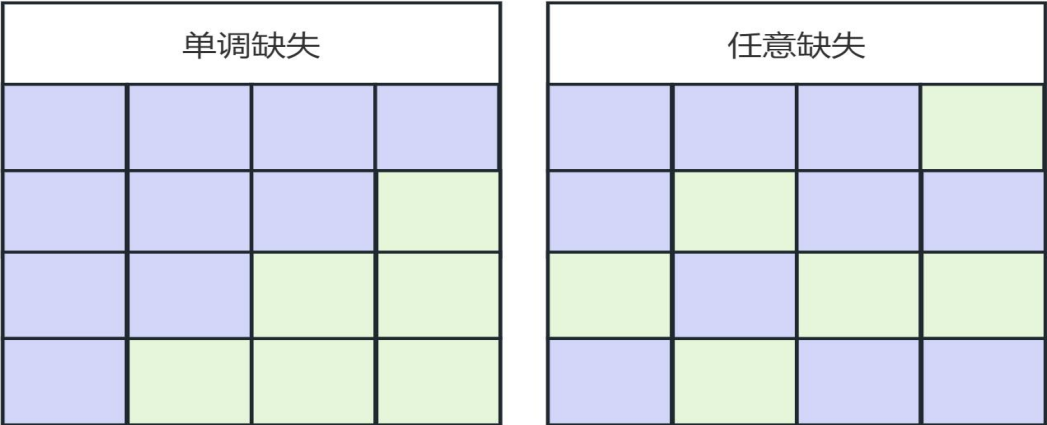
和多变量缺失。单变量缺失模式指的是在所分析的数据集中只有一个变量(特征)含有缺失值；而多变量缺失模式指的是在分析的数据集中至少有两个以上的变量(特征)含有缺失值（如图 2.4 所示）。



蓝色部分为观察值，绿色部分为缺失值

图 2.4 单变量缺失与多变量缺失简单示意图

按照缺失值变量的结构分类可以主要分为单调缺失模式和任意缺失模式。单调缺失模式，缺失值随着变量顺序的增加而呈现单调递增趋势，即某一变量之后的所有变量可能存在缺失，但之前的变量无缺失。任意缺失模式，缺失值在数据集中随机分布，无明显规律或结构，可能出现在任何变量的任意位置（如图 2.5 所示）。通过上述定义分析出文章所用的数据的缺失模式为多变量任意缺失模式。



蓝色部分为观察值，绿色部分为缺失值

图 2.5 单调缺失模式和任意缺失模型简单示意图

数据缺失机制：在上文中提到数据缺失分为数据缺失模式和数据缺失机制。数据缺失机制这个概念被分成了三类：完全随机缺失（Missing Completely at Random, MCAR）、随机缺失（Missing at Random, MAR）、非随机缺失（Not Missing at Random, NMAR）。

MCAR 机制中的缺失数据意味着缺失与研究中的特定观察或数据集中的任何其他变量无关。换句话说，具有缺失数据和没有缺失数据的观察之间没有系统性差异。

在 MAR 机制中，数据的缺失性可以根据研究中的其他变量进行预测，但不能直接从缺失数据本身预测。换句话说，缺失性的概率取决于观察数据，但不取决于具体的缺失值。

与完全随机缺失（MCAR）和随机缺失（MAR）相反，NMAR（非随机缺失）的数据表明缺失性直接与缺失观察本身的值相关。不同的数据缺失机制涉及不同的处理方法，因此我们需要判断数据集属于哪种缺失机制。按照三种机制的假定性强弱，设置零假设为 MCAR、MAR、NMAR 的顺序依次进行检验。

如何判断 MCAR，可以先按照以下步骤^[12]：

(1) 假设设定：原假设 H_0 ：数据缺失机制为 MCAR；备择假设 H_1 ：数据缺失机制非 MCAR。

(2) 判断辅助变量 x 是否符合正态分布。若符合正态分布可以使用方差齐性检验或者方差分析；若不符合正态分布可以使用 Kruskal-Wallis (K-W) 检验。

在拒绝备择假设后，可以进一步判断数据是否为 MAR 机制，与 MCAR 判断步骤相似，同样需要判断数据是否符合正态分布。若符合正态分布使用参数检验；若不符合正态分布使用非参数检验（基于 Logistic 模型的非参数化方法）^[13]。

经过初步的统计检验，研究涉及的数据集均不符合以上两种缺失机制。那么可以初步判断数据的数据缺失机制为 NMAR，但确切的结论需要根据具体场景来进一步判断。

2.3 类别分布分析

通过对各类别样本进行统计分析，如图 2.6-2.7 所示，在 6296 条样本数据中，0 类样本有 3244 条样本，占比超 50%，而其他 5 类占样本总样本不到 50%，其中 3、4、5 三类甚至占比不到总样本的四分之一。通过以上数据可以判断，该

数据集存在严重的数据类别不平衡问题,而这样的数据集在训练中可能会给模型带来偏见性,该问题是后续重点关注问题之一^[14]。

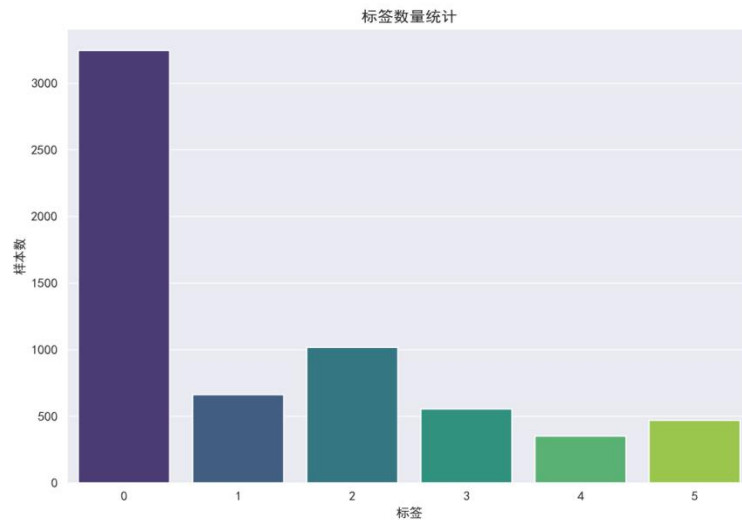


图 2.6 标签数量统计

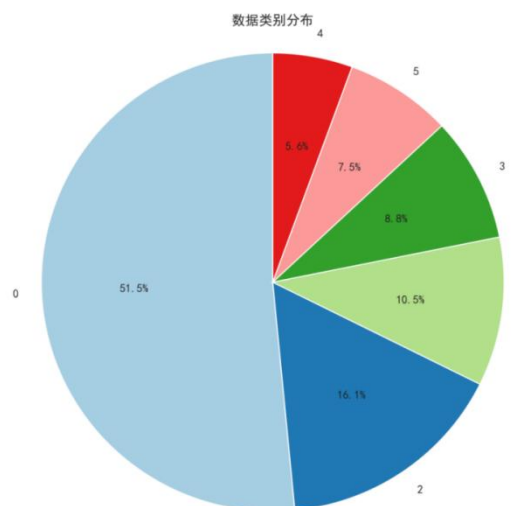


图 2.7 标签类别数量分布占比

2.4 特征分布分析

两个类别在同一特征上的分布情况如果相近,说明该特征对于这两个类别的区分能力有限。反之,如果两个类别在同一特征上的分布情况相异,那么说明该特征对于这两个类别有较强的区分能力。

分布的相近或者相异,一种方法是可以通过图片观察出来,另一种方法是可以根据统计学中的 t -statistic 和 p -value 对特征分布差异进行量化。 t -statistic 越高,说明差异越大, p -value 越小,说明观察到的结果越显著。 t -statistic 是一种用于检验统计假设的指标,它基于样本数据和总体参数之间

的差异，其计算方式涉及样本均值、总体均值和样本标准差。p-value 是用于衡量统计检验结果的一个指标。它表示在原假设为真的情况下，观察到与样本数据相关或更极端结果的概率。p 值的范围介于 0 和 1 之间，较小的 p 值表示观察到的结果在原假设下发生的概率较小，因此提供了支持拒绝原假设的证据。

如图 2.8 所示，本文统计两两类别之间，各个特征的分布差异。

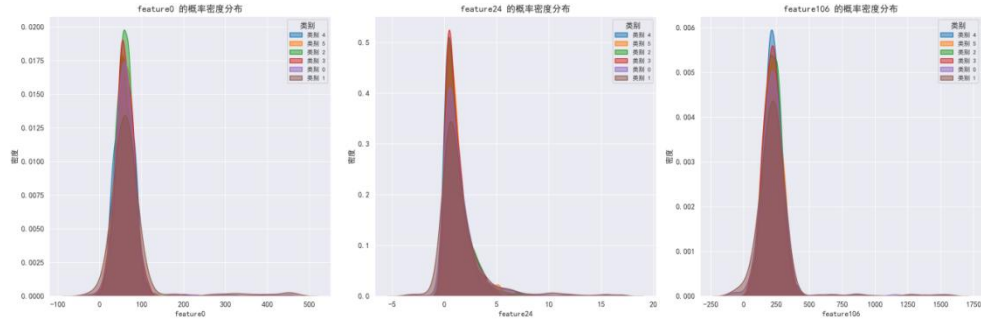


图 2.8 各类别在特征 feature0、feature24 和 feature106 上的分布情况

从图 2.9 可以看出各类别之间的可分性，并不是特别好，针对这个地方在后续数据处理和模型训练需要注意。

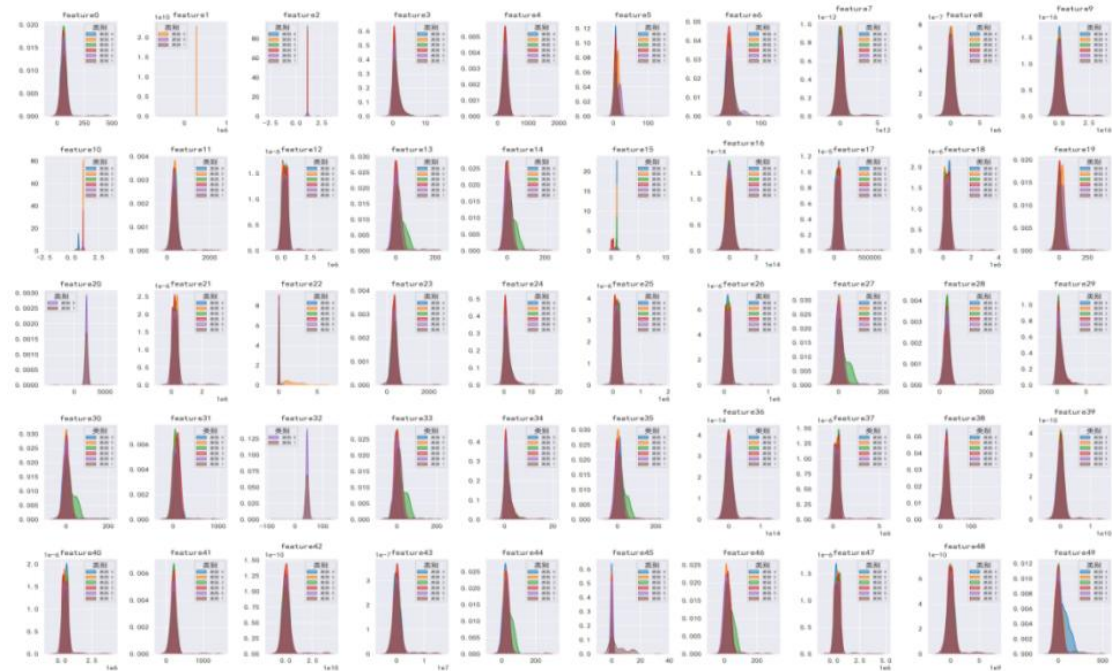


图 2.9 各类别在各个特征的分布情况（部分截图）

2.5 数据预处理

数据预处理是机器学习任务中的一个关键环节，其目标是对原始数据进行相关的数据处理操作，为后续的模型训练提供良好的数据基础。基于第一章的数据分析结论，本章将详细阐述针对训练数据集进行的各项预处理操作。

2.5.1 数据去重

数据分析阶段发现，在 10000 条原始的训练集样本中，存在高达 3704 条完全重复的样本数据。这些重复数据不仅会增加计算负担，还可能对模型的训练过程和性能评估产生错误的引导。为了保证数据的有效性和后续分析的准确性，数据预处理的首要步骤是对训练集进行去重操作。在去重后，得到有效的样本数据为 6296 条。

2.5.2 数据缺失值处理

通过数据完整性分析表明，本研究的数据集存在显著的缺失问题。具体表现为：所有样本均包含缺失值，缺失模式呈现多变量任意缺失，且缺失机制初步判断为非随机缺失（NMAR）。此外，有多达 77 个特征（覆盖率约 72%）存在不同程度的缺失，单个特征的平均缺失率约为 8.44%。

针对这种复杂的缺失情况，有如下几种方法：直接删除法、多重插补法（MCMC），以及不对缺失值做任何处理。

直接删除法，因为所有样本都含有缺失值，直接删除含有缺失值的样本这个方法明显是不行的。各特征值平均的缺失率也有 8.46%，所以删除含有缺失值的特征列这个方法是不行的。

多重插补法（MCMC），是多重插补法结合马尔科夫链蒙特卡洛树。在处理非随机缺失机制的数据集时，由于无法捕捉缺失机制与潜在数据之间的依赖关系，传统的缺失值填充法往往会面临限制。该方法基于原始数据与 EM 算法生成的初始插补值，通过数据扩增算法（DA）构建一个包含缺失数据的联合概率模型，将缺失的数值作为一个未知参数待估计，并将数据的生成过程和缺失机制在贝叶斯框架下进行显式建模^[15]。

具体实施过程中包含了交替执行的 I 步（插补步骤）和 P 步（后验步骤）：I 步从已知数据的条件分布中取样，生成缺失值的插补样本，从而达到“完整化”不完全数据的目的；P-Step 则是从贝叶斯后检验分布中，以插补后的完整数据集为基础，通过动态捕捉变量间非线性依赖关系的潜在影响和缺失机制，抽取模型参数的更新值。通过设计包含完整数据似然函数与缺失机制先验分布的联合后验分布，利用 Metropolis-Hastings 或 Gibbs 抽样算法生成马尔可夫链，在链收敛后的平稳分布中抽取有效样本完成缺失值填充。

相较于传统多重插补或极大似然估计方法，MCMC 技术通过数据扩增与贝叶斯迭代抽样，能够更灵活地处理高维非线性数据结构，尤其在 NMAR 机制下可通过显式建模缺失机制与观测数据的交互作用，避免因忽略数据生成过程导致的估计偏差。

本文使用 LightGBM 模型，这种模型是机器学习中最常用的梯度提升框架，均基于树模型（决策树），并且模型可以接受缺失值的输入。实验设计：对训练集和验证集使用同一缺失值补全方法，再使用 LightGBM 对训练集进行学习，最后在验证集上进行预测，以验证集上得到的 F1 分数作为比较的指标，不同填充方法对比如表 2.2 所示。

表 2.2 不同填充方法在 LightGBM 模型下的 F1 分数

模型	无处理	多重插补 MCMC 法
LightGBM	0.755	0.694

三种模型在无处理（直接保留缺失值）时的验证集效果均优于多重插补 MCMC 法。基于本文研究背景有如下的推断：

当某节点故障时，其自身或邻接节点的部分 KPI 指标可能因连接中断、服务不可用等一些错误情况而无法采集（例如故障节点的 featureX 缺失代表“该节点已无法上报数据”），这种“缺失”本身是故障传播的直接证据。“缺失值本身是故障特征”，直接反映节点间的依赖关系和故障传播路径，填充数据后可能会破坏这一个关键信息，为数据集引入新的噪声。

所以在后续的训练中，不对数据集进行数据缺失值填充工作。

2.5.3 数据类别不平衡处理

数据集存在着严重的数据不平衡问题，如果不对这个问题进行处理，那么在后续模型训练的过程中，模型学习可能会偏向多数类学习，造成分类边界失衡或者少数类特征学习不足等情况。

传统的处理不平衡的方法包括过采样和欠采样，但对于 LightGBM 这种树模型来说，可以在模型内部通过改变样本权重的方式来处理不平衡。具体来说，在训练过程中对于数量较少的类别的样本设置更高的权重，使其在训练中更具影响力，从而缓解数据不平衡带来的影响，平衡前后性能对比如图 2.10 所示。

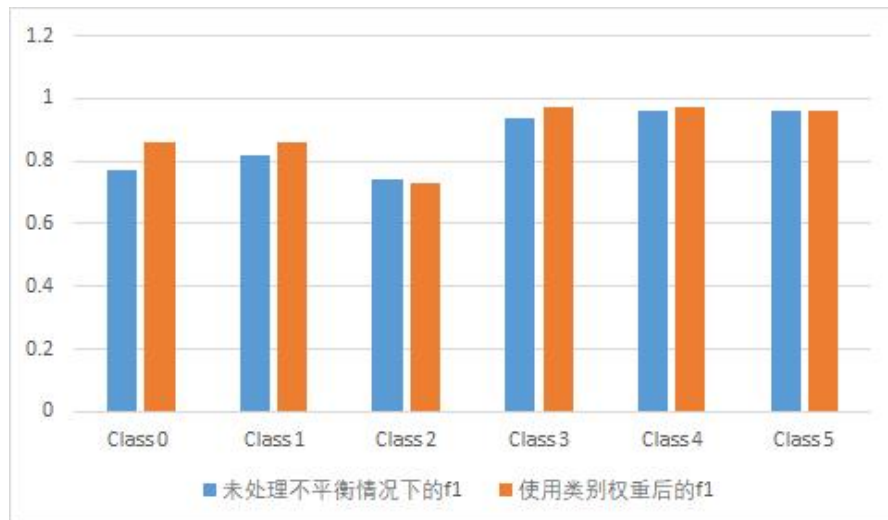


图 2.10 数据比例平衡前后的性能比较

如图 2.10 中所示，在使用类别权重平衡后 0、1、3、4 类均有提升，2、5 类略微下降，整体预测水平略有提升。

2.5.4 数据规范化

数据规范化是用于将数据转换为特定的范围或分布，以便更好地适应模型的要求。数据规范化也被称为特征缩放或特征标准化。其目的是消除特征之间的量纲差异，确保它们在相同的尺度上进行比较。这对于许多机器学习算法是必要的，因为这些算法对输入特征的范围和分布敏感。如果不对数据进行规范化，可能会导致模型性能下降或训练过程变慢。

在 LightGBM 上尝试了两个常用的数据规范化操作：最小-最大规范化（归一化），将数据缩放到一个指定的最小值和最大值之间的范围，通常是 0 到 1 之间；标准化，将数据转换为均值为 0，标准差为 1 的标准正态分布。不同规范化方法在 LightGBM 模型下的 F1 分数，如表 2.3 所示。

表 2.3 不同规范化方法的在 LightGBM 模型下的 F1 分数

模型	无处理	归一化	标准化
LightGBM	0.755	0.71	0.71

实验显示，对于 LightGBM 来说，这两种数据规范化操作都会降低模型泛化性能，分别由 0.76（不做操作）降低到了 0.71（归一化）和 0.71（标准化）。

针对这种现象有如下推断：

(1) 树模型对缩放不敏感的本质，LightGBM 是基于决策树的集成算法，决策树的分裂机制依赖于特征的相对排序（如基尼系数、信息增益），而非特征的绝

对数值。归一化/标准化会改变特征的数值范围，但不会改变特征的相对顺序或分位数分布，对树模型的分裂方向和选择影响较小。若特征间量纲差异不大，或数据中存在非线性关系，缩放可能破坏原始数据的信息结构（如极值点的区分度），导致模型难以捕捉有效模式。

(2) 预处理与 LightGBM 内置机制冲突，LightGBM 内置了对缺失值的处理（自动将缺失值视为一类），而标准化/归一化可能需要填充缺失值（如均值、中位数），引入偏差。

(3) 归一化（如 Min-Max 缩放）对异常值敏感，可能将少数异常值的影响扩散到整个特征范围，导致缩放后的数据分布偏离真实模式。

(4) 标准化（Z-score）假设数据服从正态分布，若数据实际分布非正态（如偏态），缩放后均值和方差的意义被扭曲，特征的区分度下降。

综上所述，对于所选用的模型 LightGBM，模型最关注的是特征的相对顺序而非值，因此要尽可能地保留数据的原始形状，选择不使用数据规范化。

2.5.5 特征工程

在机器学习训练中，并不是所有特征都是有价值的，特征工程旨在从原始数据中提取、构造或选择对模型预测最有用的特征，消除冗余信息，提高训练效率、减少计算开销、避免模型发生过拟合。在本研究中，考虑到数据集的特性和后续模型的选择，特征工程主要聚焦于以下几个方面：

(1) 常数特征移除

在数据分析阶段，我们对所有（去重后）训练样本中各个特征的取值进行了检查。分析发现，feature57, feature77, feature100 这三个特征在所有样本中均呈现完全相同的值。这类特征被称为常数特征或零方差特征。由于它们在不同的样本数据之间没有任何变化，因此不包含任何能够区分不同样本或不同故障类别的信息，对模型的学习和预测任务没有任何贡献，反而可能在某些算法中引入不必要的计算或者噪声。因此，作为特征工程的第一步，我们直接将这三个常数特征从数据集中移除。移除后，剩余待分析的特征数量为 104 个。

(2) 特征选择策略与 LightGBM 的内在机制

对于特征选择，即从剩余的 104 个特征中筛选出对故障诊断最有价值的特征子集，常见的特征方法大致可以分成三类^[16]，如图 2.11 所示：

过滤式方法（Filter Methods）。这类方法独立于具体的机器学习算法，通

通过对特征之间的关系进行统计分析，来评估特征的重要性。方差选择和单变量特征选择属于此类方法。它们基于特征自身的统计指标或与目标变量之间的统计关系来选择特征。

包装式方法（Wrapper Methods）。这类方法使用特定的机器学习算法作为特征选择的基础，将特征选择的过程与模型的性能表现相结合。通过反复训练模型来选择最佳特征子集。其中递归特征消除（RFE）是典型的封装型方法，它使用模型的性能作为评估指标来选择特征。

嵌入型方法（Embedded Methods）。这类方法，特征选择作为机器学习算法的一部分，将特征选择过程融入学习算法的训练过程中。算法在模型构建的同时自动进行特征权衡和选择。例如，Lasso 回归通过 L1 正则化将不重要特征的系数压缩至零；基于树的集成模型（如本研究后续采用的 LightGBM、XGBoost、CatBoost）在构建决策树的每个节点分裂时，会根据信息增益、基尼不纯度等准则选择最优的切分特征，这本身就是一种有效的特征选择机制。

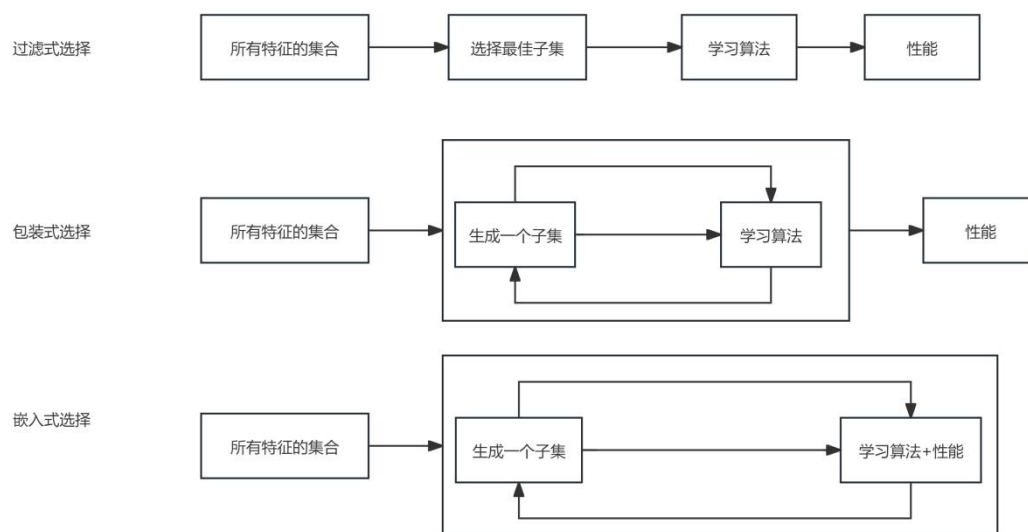


图 2.11 三种主流的特征选择方法

考虑到文章选用的 LightGBM 模型属于基于树的集成模型，其在构建决策树的过程中，会根据信息增益（或其他指标）自动评估每个特征在每次分裂时的重要性，并倾向于优先使用更重要的特征。

这种机制相当于内嵌了特征选择的功能，模型能够自动学习忽略或减少不重要特征的影响。

(3) 显式特征筛选实验

尽管 LightGBM 具有内在的特征选择能力，我们仍然希望对特征重要性有一个直观的了解，并验证是否有必要进行额外的显式特征筛选。

为了验证是否需要进行显式的特征选择，尝试使用一种“二阶段特征筛选”策略：首先训练一个 LightGBM 模型，然后根据模型输出的特征重要性排序，移除一部分重要性较低的特征，再用筛选后的特征重新训练模型。

实验结果如图 2.12 所示。

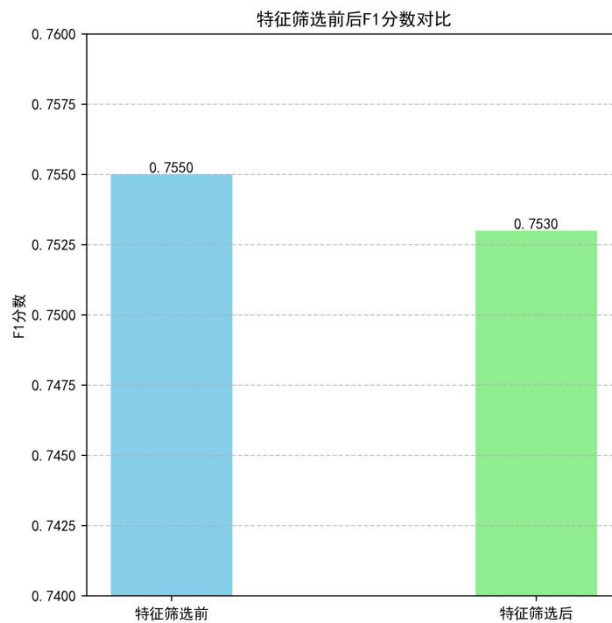


图 2.12 显示特征删除前后分数对比

实验结果表明，这种显式的特征删除并未带来性能提升，有时甚至略有下降。这进一步证实了 LightGBM 模型自身处理特征重要性的能力。

因此，本文中不进行额外的显式特征选择步骤，而是依赖 LightGBM 模型在训练过程中隐式地完成特征选择。

2.6 小结

综上所述，根据本章节数据分析的结果，可知在训练数据集中 10000 份样本存在 3704 条重复数据。数据集为多变量任意缺失模式，缺失值机制初步判断为 NMAR 机制，所有样本数据均存在缺失值，有 77 个特征存在着缺失值，30 个特征没有缺失值，覆盖率约 72%，缺失值特征的评价缺失率在 8.44%。按类别分类发现数据集存在严重的数据不平衡问题，其中 0 类样本占比超过 50%，并且各类别之间的可分性并不是特别良好。

同时确定了一套针对性的数据预处理策略：删除重复样本、移除常数特征、

不进行缺失值填充、不进行数据规范化，并利用模型内置机制处理类别不平衡。这些决策均基于对数据特性和模型行为的深入理解及实验验证，为后续构建高性能的故障诊断模型奠定了坚实的基础。

第三章 模型训练及优化

3.1 LightGBM 模型

LightGBM (Light Gradient Boosting Machine)，是一种用于解决分类和回归问题的梯度提升机 (Gradient Boosting Machine, GBM) 算法^[17]。

模型是一种基于树的集成学习方法，采用了梯度提升技术，通过将多个弱学习器组合成一个强大的模型。相比于传统的 GBDT 算法，LightGBM 模型引入了基于 Histogram 的决策树算法、GOSS (基于梯度的单边采样技术) 和 EFB (互斥特征捆绑技术) 和 EFB 三种关键的技术，其优势主要体现在加快训练速度、减小内存消耗这两个方面。

3.1.1 基于直方图的决策树构建

传统的 GBDT 生成树结构时要遍历所有的特征的所有的可能分裂点来决定最好的选哪个特征以及在哪一维度分裂出最佳树，在面对大量的数据和特征维度时，十分费时。

LightGBM 使用基于直方图的方法，将连续型特征值映射到事先确定好数量的 K 个区间，并记录下每个特征的分布直方图，在寻找最优分割点的过程中，只需要考虑 k 个区间边界的比选，不需要比对全部原始数值点，因此大大降低计算量，加快了学习的速度；直方图同时能够很好地控制内存占用，因为其只需要存储离散化的区间的部分而不是完整的精确浮点数。

3.1.2 梯度单边采样

这种方法是在保持正样本比例的前提下抽取每批中大部分负样本、少部分梯度绝对值较大的正样本。

梯度提升算法下的每个样本对应的梯度值不一样，一般来说，梯度越大，说明越能反映当前模型在这个样本上的预测误差，而梯度值大的样本一般会对后续模型进行较大改进，梯度值小且模型已经能够较好地拟合样本，对应后续模型对其改进微乎其微。

GOSS 策略的精髓在于：在更新下一个弱学习器前，将所有的梯度大的样本都完整地留下，从剩下的梯度小的样本里面随机采样即可。利用这种方法，在不损失太多精度的前提下，用很少的一部分样本就完成了整个模型的训练，大大节省了时间成本。由于大部分“学习尚不充分”的样本很少被考虑，所以 G-Assad

同样在一定程度上提高了整个训练过程的速度。

3.1.3 互斥特征捆绑

在高特征维的数据中，存在很多特征项是比较稀疏的，而且某些特征项之间可能会存在排他性，即特征项中不可能出现某两个或多个特征项的值均为非 0 的情况。典型的例子是，在自然语言处理 (NLP) 中采用的独热编码 (One-Hot Encoding) 所获得的不同单词的特征向量就属于这种情况。

EFB 是基于这样一种特性，即特征之间互斥，把所有符合互斥条件的特征“聚合”成一个“特征包”，对于构建直方图的方法，就可以把这个组合在一起的“特征包”当成是一整套处理，放进同一个数据结构中去实现。这样，特征空间的维度就减少了，计算复杂度降低了，这样就可以提高训练速度，并且对模型的精度影响极小。

3.2 评价指标

Macro_F1 值，是综合衡量分类模型在多个类别上性能的指标。它综合考虑了模型在各个类别上的精确率 (P) 和召回率 (R)。通过调和平均的方式，避免了因类别不平衡等因素导致的评估偏差，更全面地反映模型整体分类能力。其计算方式如下所示：

$$\text{macro_P} = \frac{\sum_{i=1}^n P_i}{n} \quad (\text{公式 3.1})$$

$$\text{macro_R} = \frac{\sum_{i=1}^n R_i}{n} \quad (\text{公式 3.2})$$

$$\text{macro_F1} = \frac{2 * \text{macro_P} * \text{macro_R}}{\text{macro_P} + \text{macro_R}} \quad (\text{公式 3.3})$$

其中 Macro_P 为平均精确率，Macro_R 为平均召回率， P_i 为每一个类别的精确率， R_i 为每一个类别的召回率， n 为类别总类。排行得分为 $\text{Macro_F1} * 100$ ，得分越高说明模型的综合性能越好。

3.3 实验设计

该项目采用 LightGBM 作为基础模型进行训练，设计实验并观察模型性能。

3.3.1 数据准备

加载数据集，将数据集中的特征与标签进行划分，移除方差小于 0.1 的特征，特征，对特征进行筛选，减少数据维度。将数据集按照 7: 3 的比例划分为训练集和验证集。

3.3.2 模型训练

选择 LightGBM 模型进行训练。设置模型参数包括目标函数、评估指标、类别数量，其余参数为默认参数。使用早停策略，如果验证集的性能在设置的阈值之内没有提升，则提前停止训练。使用训练集对模型进行训练，并在验证集上进行验证。

3.3.3 模型评估

根据 3.2 小节的评价指标，定义评估函数，加载测试数据集，并使用测试数据集对模型性能进行评估。

3.4 结果展示

使用测试集对 LightGBM 模型进行验证，验证结果得到各类别的 F1 分数，结果如表 3.1 各类别的预测值的精确度、召回率和 F1 分数所示。

根据表 3.1 结果展示可以计算出 Micro_F1 为 0.75。

表 3.1 各类别的预测值的精确度、召回率和 F1 分数

故障类别	Precision	Recall	F1
0	0.5639	0.7273	0.6352
1	0.5381	0.6807	0.6011
2	1.0000	0.4678	0.6375
3	0.9774	0.7692	0.8609
4	0.7525	0.9551	0.8418
5	0.9868	0.9259	0.9554

3.4.1 整体性能评估

类别 4 (F1=0.8418) 和类别 5 (F1=0.9554) 表现最佳，模型对这两类的识别能力较强。

类别 2 (F1=0.6375) 和类别 1 (F1=0.6011) 表现最差，模型在识别这两类时存在明显困难。

类别 0、1、4 的 Recall 高于 Precision，说明模型更倾向于将样本预测为这些类别，但可能存在较多误判。

类别 2 的 Precision 为 1.0000，但 Recall 仅为 0.4678，表明模型对该类的预测非常保守，虽然预测准确但漏检严重。

3.4.2 类别间性能对比

类别 5 的 Precision (0.9868) 和 Recall (0.9259) 都很高, 说明模型对该类的区分能力强, 特征明显且与其他类别差异大。

类别 3 的 Precision (0.9774) 较高, 但 Recall (0.7692) 有提升空间, 可能存在部分样本与其他类别混淆。

类别 2 的高 Precision 和低 Recall 表明模型对该类的特征把握较准, 但可能因训练样本不足或特征不典型导致漏检。

类别 1 的 Precision 和 Recall 都较低, 可能是该类与其他类别存在较多重叠特征, 模型难以区分。

3.5 参数调优

根据上一个小节的结果, 测试集在模型上的 F1 分数为 0.75, 为了进一步提升模型的泛化能力, 需要寻找模型的最佳超参数, 进一步优化模型。

对于 LightGBM 模型, 参数优化涉及多个参数, 包括: 学习率(learning_rate)、树的最大深度(max_depth)、节点分裂的最小增益(min_split_gain)、每个树的叶子节点数(num_leaves)、数据采样比例(subsample)、L1 正则化系数(subsample)、L2 正则化系数(reg_lambda), 这 7 个参数。

每一个超参数的选择都是多样的, 如果仅通过人工的方法筛选, 筛选效率非常低, 并且有可能会错过最佳超参数。

对于超参数优化算法选择混合优化策略, 单一的优化算法往往存在多种限制, 例如无法同时兼顾精度和计算时间。所以在这里采用遗传算法+贝叶斯优化+网格搜索的混合优化算法^[18]。

3.5.1 遗传算法

遗传算法 (Genetic Algorithm, GA) 是模拟了生物进化中的遗传和自然选择机制, 通过对候选解进行遗传操作和适应度评估来搜索问题的最优解。

它通过维护一个种群 (Population) 来代表问题的解空间, 其中每个个体 (Individual) 都表示一个候选解。它采用遗传操作, 包括选择、交叉和变异, 以产生新的个体, 并使用适应度函数评估个体的优劣程度。在每一代 (Generation) 中, 根据个体的适应度进行选择操作, 优秀的个体有更高的概率被选择作为父代参与交叉和变异操作, 从而产生下一代个体。通过不断地迭代, 使其适应度逐步提高, 最终找到问题的最优解或接近最优解的解。

遗传算法的优势在于能够在较大的搜索空间中广泛探索不同的参数组合,通过交叉和变异操作生成多样化的解,算法执行流程如图 3.1 所示。

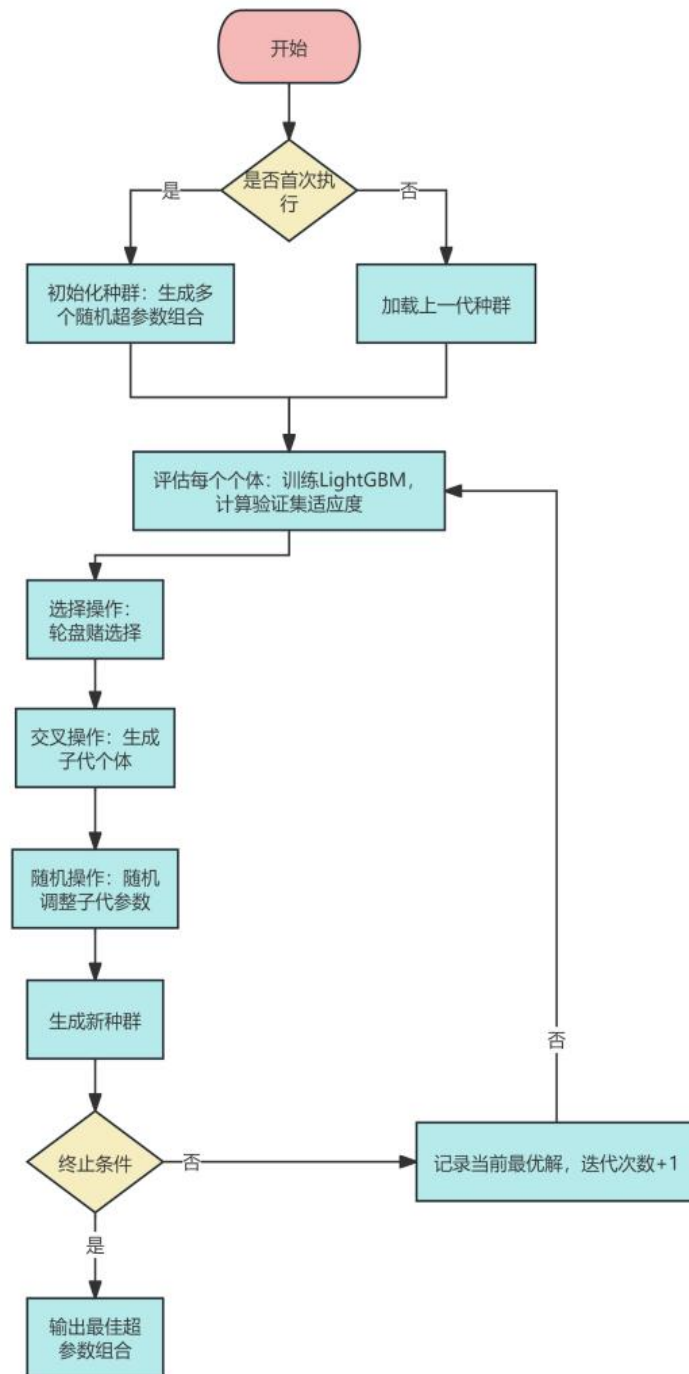


图 3.1 遗传算法流程

用遗传算法对模型超参数进行优化,使用遗传算法进行超参数搜索 30 次迭代过程,结果如图 3.2 所示:

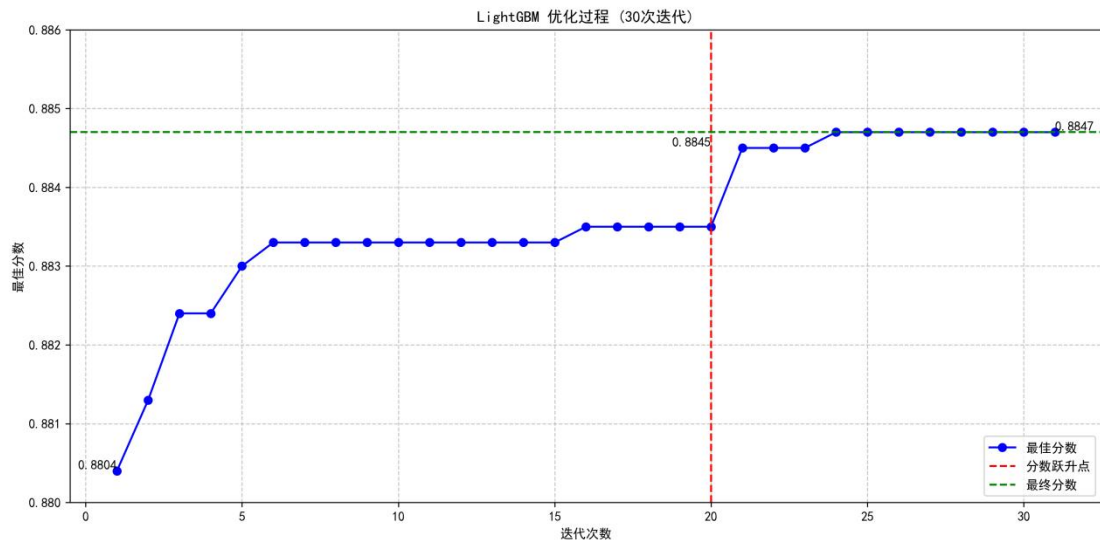


图 3.2 遗传算法在 LightGBM 迭代 30 次的优化情况

实验表明，遗传算法确实能够稳步地提升模型性能，在后期会进入稳态，不再变化，最终 F1 分数在 0.8845。它的优点是比较稳定，能够稳步提升，缺点在于它有陷入局部最优解的风险，超参数集合如表 3.2 所示。

表 3.2 LightGBM 模型（遗传算法）参数表

参数	参数范围	最佳参数
num_leaves	[20, 40]	30
learning_rate	[0.01, 0.1]	0.04
max_depth	[3, 9]	7
reg_alpha	[0, 1]	0.593
reg_lambda	[0, 1]	0.730
subsample	[0.5, 1]	0.870
min_split_gain	[0.1, 0.5]	0.116

3.5.2 贝叶斯优化

适用于处理高维、非凸的参数空间，尤其适合 LightGBM 这类复杂模型。其思想是通过高斯过程（Gaussian Process）建模目标函数的后验分布，利用采集函数（如期望改进 EI、概率提升 PI）平衡探索（Exploration）与利用（Exploitation），逐步逼近最优参数组合。

贝叶斯优化的核心是利用高斯过程对目标函数 $f(\theta)$ （如 Macro_F1）进行建模。高斯过程是一种随机过程，假设任意有限个函数值服从联合正态分布，其由均值函数 $m(\theta)$ 和协方差函数（核函数） $k(\theta, \theta')$ 完全定义。

给定一组历史观测数据 $D = \{(\theta_i, y_i)\}_{i=1}^n$ ，其中 θ_i 为超参数向量， $y_i = f(\theta_i)$ 为目标函数值，高斯过程通过以下步骤构建后验分布：

- 均值函数：通常设为常数或零均值，即 $m(\theta) = \mu$ 。
- 协方差函数：采用平方指数核（SE 核）描述参数空间的相似性： $k(\theta, \theta') = \sigma^2 \exp(-\frac{\|\theta - \theta'\|^2}{2l^2})$ 为信号方差， l 为长度尺度，控制参数空间的平滑度。

基于贝叶斯定理，观测数据 D 条件下的目标函数在新参数 θ^* 处的后验分布为正态分布：

$$f(\theta^*)|D \sim N(\mu_D(\theta^*), \sigma_D^2(\theta^*)) \quad (\text{公式 3.4})$$

其中，后验均值和方差通过以下公式计算：

$$\mu_D(\theta^*) = k(\theta^*)^T K^{-1} y \quad (\text{公式 3.5})$$

$$\sigma_D^2(\theta^*) = k(\theta, \theta') - k(\theta^*)^T K^{-1} k(\theta^*) \quad (\text{公式 3.6})$$

此处 K 为历史数据的协方差矩阵， $k(\theta^*)$ 为新参数与历史数据的协方差向量。

贝叶斯优化使用采集函数（Acquisition Function）在“探索未知区域”和“利用已知优解”之间权衡，选择下一个待评估的参数点 θ_{next} 。常用采集函数包括：

（1）期望改进：在当前最优解 y_{best} 的基础上衡量期望提升度，公式为：

$$EI(\theta) = \int_{-\infty}^{y_{\text{best}}} (y_{\text{best}} - y) p(y|\theta, D) dy \quad (\text{公式 3.7})$$

其中 $p(y|\theta, D)$ 是目标函数在 θ 处的后验概率密度。当后验分布为 $N(\mu, \sigma^2)$ 的时候， EI 可以简化为：

$$EI(\theta) = \sigma \left(\left(\frac{\mu - y_{\text{best}}}{\sigma} \right) \phi\left(\frac{\mu - y_{\text{best}}}{\sigma}\right) + \phi\left(\frac{\mu - y_{\text{best}}}{\sigma}\right) \right) \quad (\text{公式 3.8})$$

其中 $\phi(\frac{\mu - y_{\text{best}}}{\sigma})$ 为标准正态分布的累积分布函数。

（2）概率提升：计算新参数点优于当前最优解的概率，公式为：

$$PI(\theta) = \phi\left(\frac{\mu - y_{\text{best}}}{\sigma}\right) \quad (\text{公式 3.8})$$

采集函数的选择取决于问题特性： EI 在多数场景下表现均衡， PI 则适用于更注重探索的场景。

根据遗传算法寻找的超参数空间，重新定义超参数的取值范围，使用贝叶斯优化算法，迭代 30 轮次，实验结果如图 3.3 所示：

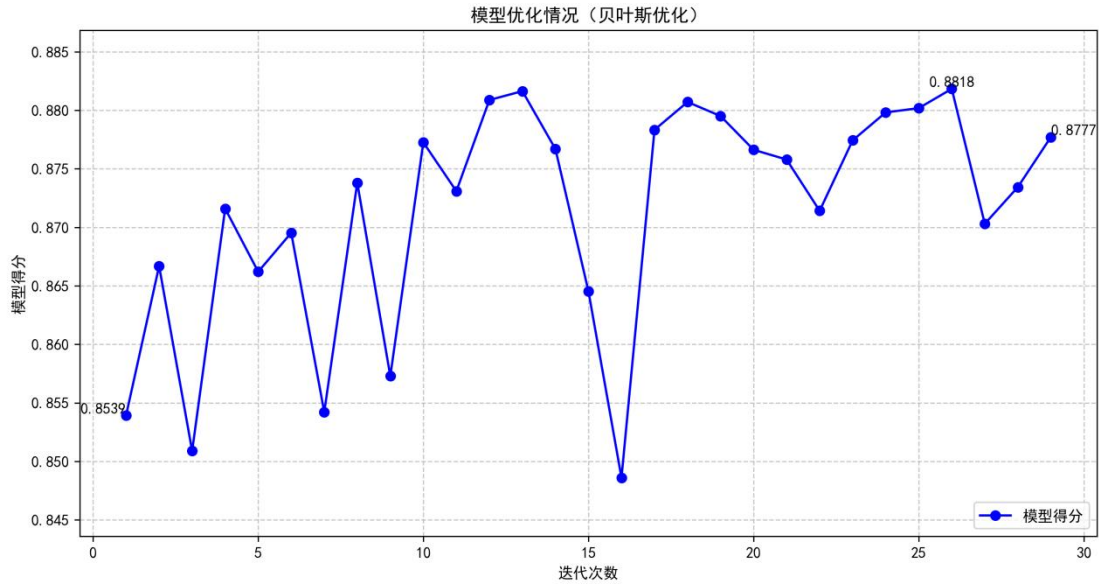


图 3.3 贝叶斯优化算法在 LightGBM 迭代 30 次的优化情况

通过贝叶斯优化算法，在给定超参数空间进行搜索，模型得分在 0.845-0.885 之间波动，最好的成绩是 0.8818，贝叶斯优化搜索超参数集合如表 3.3 所示。

表 3.3 LightGBM 模型（贝叶斯优化）参数表

参数	参数范围	最佳参数
num_leaves	[25, 35]	30
learning_rate	[0.04, 0.09]	0.06
max_depth	[3, 7]	5
reg_alpha	[0.5, 1]	0.7
reg_lambda	[0.7, 1]	0.9
subsample	[0.5, 0.8]	0.7
min_split_gain	[0.1, 0.5]	0.2

3.5.3 网格搜索

网格搜索是一种基于穷举的参数调优方法，通过定义参数网格并遍历所有组合，选择在验证集上性能最佳的参数，网格搜索最佳参数如表 3.4 所示。

表 3.4 LightGBM 模型（网格搜索）参数表

参数	参数范围	最佳参数
num_leaves	[29, 30, 31]	31
learning_rate	[0.04, 0.05, 0.06]	0.05

续表 3.4 LightGBM 模型（网格搜索）参数表

参数	参数范围	最佳参数
reg_lambda	[0.7, 0.8, 0.9, 1]	0.8
subsample	[0.5, 0.6, 0.7]	0.5
min_split_gain	[0.2, 0.3, 0.4, 0.5]	0.3
max_depth	[3, 4, 5]	3
reg_alpha	[0.7, 0.8, 0.9, 1]	0.8

3.5.4 结果分析

在经过使用混合算法对 LightGBM 的 7 个超参数进行搜索，最终得到最优超参数集合，使用优化后的模型重新训练，在测试集上的 F1 分数由原来的 0.75 上涨到 0.849，调参前后对比如表 3.5 所示。

表 3.5 LightGBM 模型调参前后对比

LightGBM	Macro_Precision	Macro_Recall	Macro_F1
调参前	0.75	0.75	0.75
调参后	0.849	0.849	0.849

3.6 模型融合

为了进一步提高分类水平，在这里引入模型融合策略，考虑到单一的 LightGBM 模型可能在某些方面存在着局限性，选择使用加权投票融合策略^[19]。结合不同的模型进行优势互补。

选择其他基模型需要满足以下条件：能够处理缺失值和类别不平衡、与 LightGBM 模型可以互补。

最终选择 XGBoost 和 CatBoost 两个模型。这两个模型与 LightGBM 一样都是基于梯度提升决策树（GBDT）的模型。

XGBoost 模型与 LightGBM 模型在分裂策略和优化目标是存在着差异。XGBoost 基于预排序算法和深度优先分裂策略，可以在一定程度上弥补 LightGBM 模型的过拟合风险，提高模型的鲁棒性^[20]。

CatBoost 模型是专门为高基数类别特征设计的模型。CatBoost 通过“有序提升”减少过拟合风险，补充 LightGBM 在类别特征处理上的不足，提升模型性能^[21]。

下面对 XGBoost 和 CatBoost 两个模型分别进行训练，并用测试集进行评估，

评分如下表 3.6 所示。

表 3.6 XGBoost 和 CatBoost 的评估结果

模型	Macro_Precision	Macro_Recall	Macro_F1
XGBoost	0.772	0.772	0.772
CatBoost	0.77	0.77	0.77

接下来使用混合优化策略,对两个模型进行超参数搜索,进一步优化模型分类水平, XGBoost 最佳参数如下表 3.7 所示。

表 3.7 XGBoost 最佳参数

参数	最佳参数
n_estimators	200
learning_rate	0.05
max_depth	3
reg_alpha	0.5
reg_lambda	4
min_child_weight	1
colsample_bytree	0.7

CatBoost 最佳参数如 3.8 表所示。

表 3.8 CatBoost 最佳参数

参数	最佳参数
iterations	2000
learning_rate	0.01
depth	3
l2_leaf_reg	0.140
bagging_temperature	0.742
min_data_in_leaf	30

通过使用混合优化策略寻找最佳超参数,得到最优超参数集合,使用新的超参数集合重新对模型进行训练,如表 3.9 所示, XGBoost 模型 F1 评分从 0.772 提升到了 0.842, CatBoost 模型 F1 评分从 0.77 提升到了 0.835。

表 3.9 XGBoost 和 CatBoost 使用超参数优化后评分

模型	Macro_Precision	Macro_Recall	Macro_F1
XGBoost	0.842	0.842	0.842
CatBoost	0.835	0.835	0.835

接下来进行加权投票模型融合，首先获取每个基模型对于测试集的概率类别预测；将各个模型的概率预测乘以权重后求和；对加权后的概率分布取最大值索引作为最终预测类别，经过多次迭代，得到最优的权重集合，加权投票融合策略流程如图 3.4。

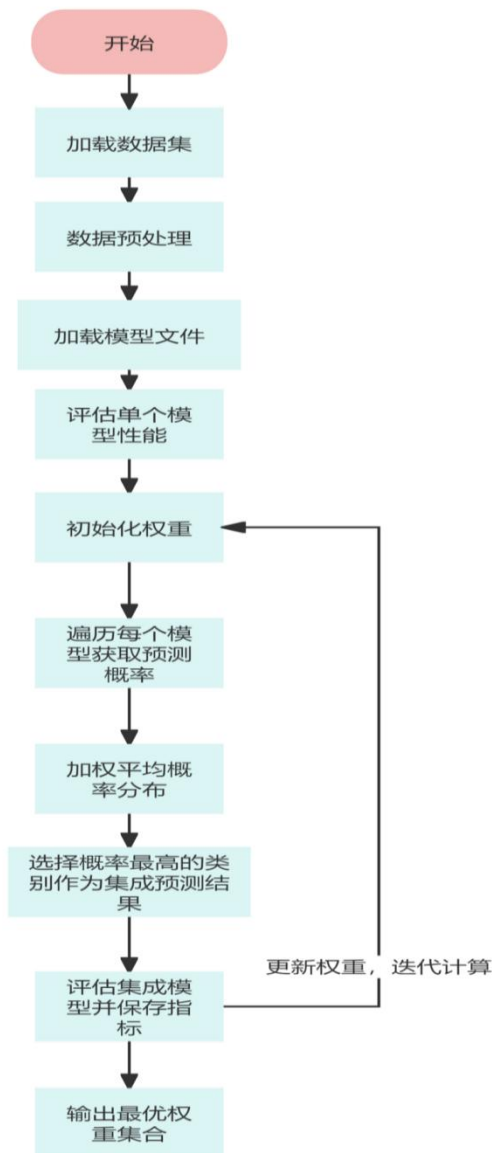


图 3.4 加权投票融合策略流程图

最终通过模型融合，在测试集上进行评估，结果如表 3.10 所示，F1 分数为 0.855，比前与前三个模型相比得到了提升。

表 3.10 三种模型以及模型融合的评估结果

模型	Macro_Precision	Macro_Recall	Macro_F1
LightGBM	0.849	0.849	0.849

续表 3.10 三种模型以及模型融合的评估结果

模型	Macro_Precision	Macro_Recall	Macro_F1
XGBoost	0.842	0.842	0.842
CatBoost	0.835	0.835	0.842
加权投票融合模型	0.855	0.855	0.855

使用加权投票融合在各类别上的 F1 分数如图 3.5 所示。

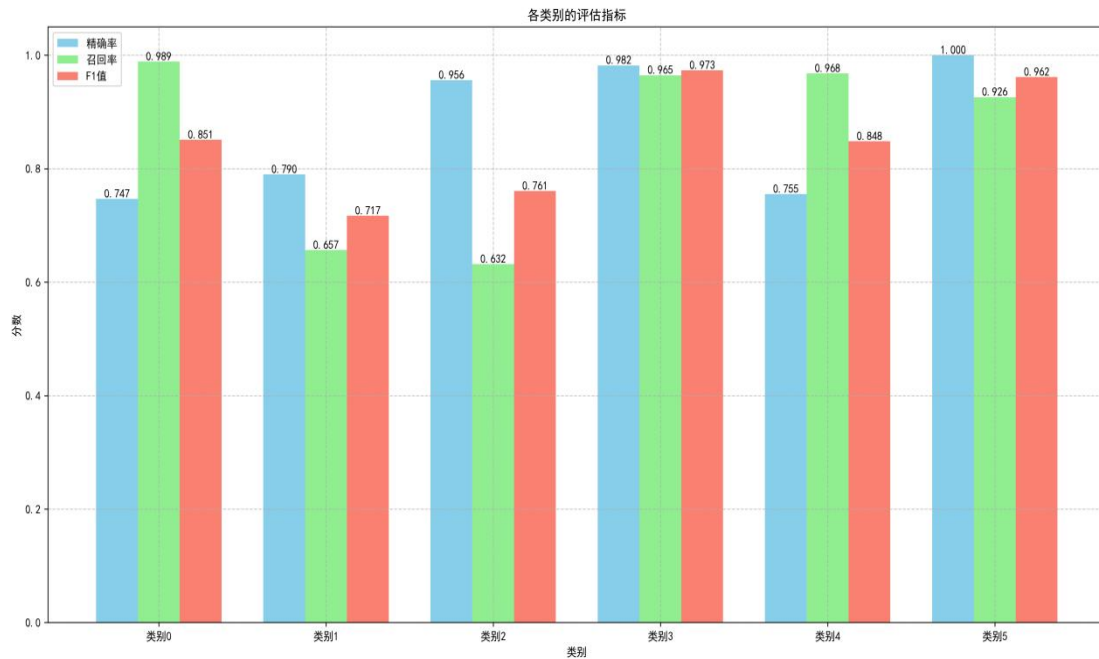


图 3.5 加权投票融合在各类别上的 F1 分数

3.7 小结

本章围绕分布式系统故障诊断模型的训练与优化展开研究，基于 LightGBM 模型构建了训练框架，并通过参数调优及模型融合等策略提升分类性能。具体结论如下：

基础模型性能分析：初始 LightGBM 模型在处理缺失值和类别不平衡问题时，利用模型本身的机制保留原始数据特征，避免使用其他填充方法引入噪声。在未进行显式特征选择的情况下，模型对故障类别 3、4、5 表现出较强识别能力，但对类别 1、2 的区分存在不足，反映出部分类别可分性不良的问题。

超参数优化效果：采用遗传算法+贝叶斯优化+网格搜索的混合优化策略，针对 LightGBM 的 7 个关键参数进行调优。实验表明，通过混合优化策略，通过遗传算法在给定范围内全局搜索，避免陷入局部最优。然后使用贝叶斯优化和网格搜索在遗传算法搜索的结果下进一步精确搜索，找到最佳超参数。最终将

Macro_F1 从 0.75 提升至 0.849，提高了模型的泛化能力。

模型融合策略优势：引入 XGBoost 和 CatBoost 作为基模型，与 LightGBM 形成互补。XGBoost 的预排序分裂策略和 CatBoost 对高基数类别特征的处理能力，有效弥补了 LightGBM 在过拟合控制和类别特征处理上的不足。通过加权投票融合，最终模型的 Macro_F1 提升至 0.855，较单一模型性能进一步优化，表明多模型融合能够通过优势互补提升整体分类精度。

综上所述，本章通过系统性的模型训练与优化，构建了高性能的故障诊断模型，为后续分布式系统故障诊断系统的实现奠定了算法基础。未来可进一步探索更复杂的融合策略或结合深度学习模型，进一步提高分类水平。

第四章 系统设计

4.1 需求分析

4.1.1 应用场景

系统应用在分布式系统的故障诊断，需处理节点故障时产生的异常 KPI 指标数据（含 107 个特征），支持对 6 类故障（含无故障类别）进行分类，适用于分布式系统运维等场景。

4.1.2 系统角色

运维工程师：需要通过系统快速定位故障类别，分析故障传播路径，优化系统稳定性。

4.1.3 功能需求

（1）文件上传和下载功能：对于基于机器学习的分布式系统故障诊断系统，必须提供用户上传训练数据集和测试数据集的入口，同时提供训练后的模型下载功能。

（2）数据预处理：系统需要支持对用户上传的文件进行自动化的预处理功能。

（3）模型训练：系统要能提供模型训练所需参数设置，并进行模型训练，以满足用户训练所需模型。

（4）数据可视化：系统需要提供对于数据集、训练性能指标的数据可视化功能，方便用户查看数据的情况。

（5）故障预测：系统需要支持用户上传数据集，使用选定的模型对数据集进行预测。

4.1.4 性能需求

系统必须确保以下主要性能要求：

- （6）B/S 体系结构。以 Web 客户端提供用户 UI 界面；
- （1）UI 界面简洁美观；
- （2）页面在使用的时候能够及时响应，快速加载内容；
- （3）系统运行流畅、稳定、可靠；

4.2 系统设计

4.2.1 系统总体功能设计

系统是一个基于 Flask 框架的 Web 应用程序，主要分为三大模块，分别是数据可视化模块、训练模块和预测模块。

系统总体功能设计如图 4.1 所示。

(1) 数据可视化模块：支持用户上传数据集文件（格式为 csv 格式），上传文件后，根据后端设定的数据可视化功能代码，将数据集相关数据信息进行可视化，并返回前端供用户查看。

(2) 模型训练模块：该模块支持用户上传自己的数据集（格式为 csv），用户可以调整模型参数，点击提交后系统会自动根据后端实现的训练代码，训练模型。训练结束后支持用户将训练好的模型下载。

(3) 预测模块：该模块支持用户上传自己的数据集（格式为 csv），用户可以选择模型或者导入自己训练好的模型到系统中，使用模型对数据集进行预测，并返回预测结果。

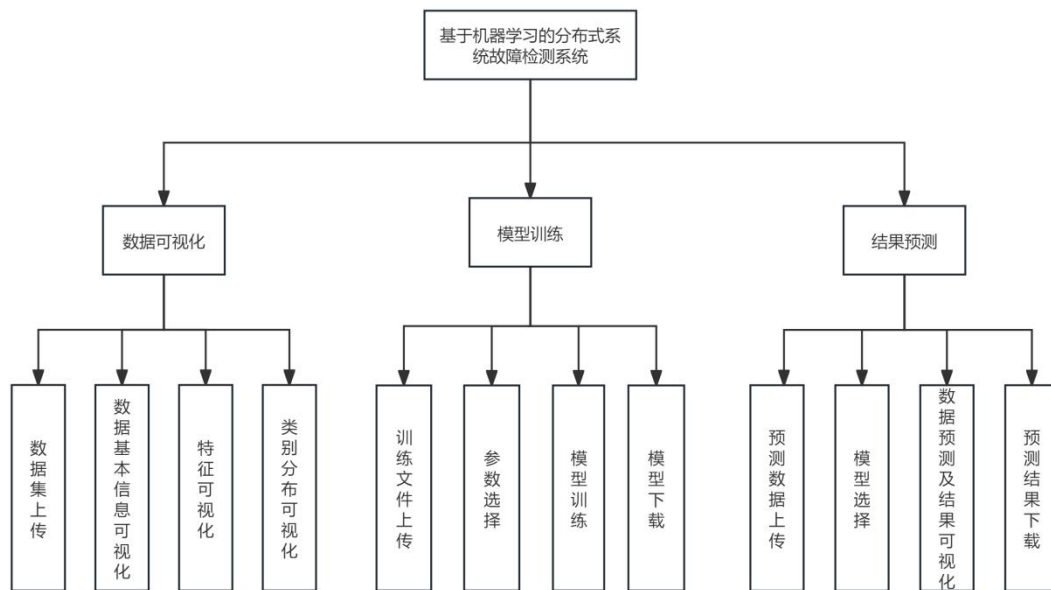


图 4.1 系统总体功能结构图

4.2.2 数据库设计

为了有效地对系统产生的数据进行管理，如用户上传的数据集、训练生成的模型、模型参数等数据信息，系统采用关系型数据库 MySQL 对生产数据进行存储和管理。关系型数据库具有结构化数据存储、事务一致性、数据完整性高等优点，能够很好地满足本系统的需求。

数据库整体设计如图 4.2 所示。

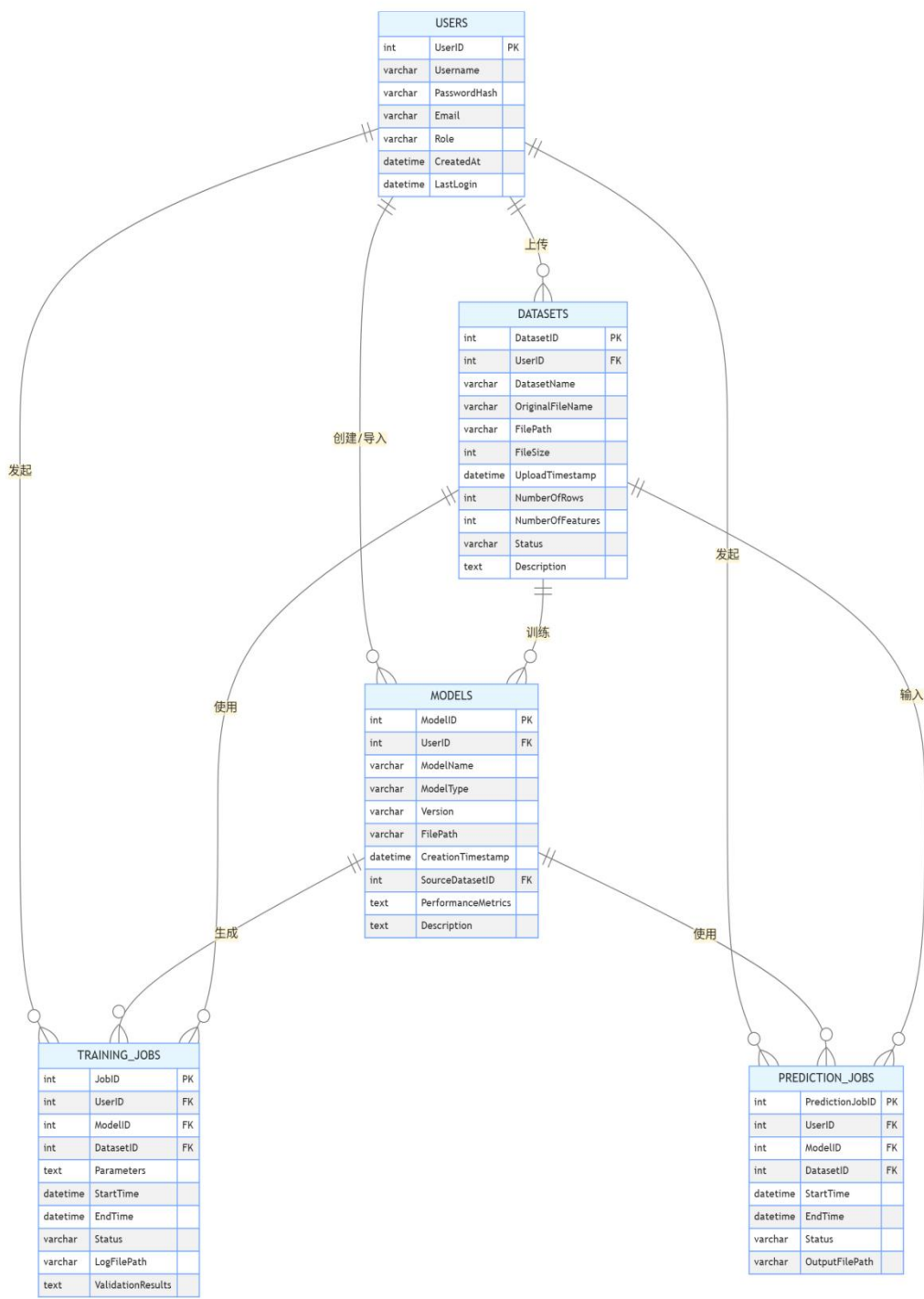


图 4.2 数据库设计

数据库主要包含以下几种数据表：

(1) 用户表：包含用户 ID、用户名、密码等属性，主要功能是存储系统用户信息。表结构如下表 4.1 所示。

表 4.1 用户表主要字段和字段描述

主要字段	描述
UserID	主键, 用户 ID
Username	用户名
PasswordHash	加密后的密码
Email	邮箱
Role	角色, 例如: 运维工程师
CreatedAt	账户创建时间
LastLogin	上次登录时间

(2) 数据集表: 包含数据集 ID、数据集名称、文件路径以及用户 ID。主要功能是管理用户上传的用于模型训练或预测的原始数据集。表结构如下表 4.2 所示。

表 4.2 数据集表主要字段和字段描述

主要字段	描述
DatasetID	主键, 数据集 ID
UserID	外键, 关联用户表, 记录上传者
DatasetName	数据集名称, 用户自定义
OriginalFileName	原始文件名
FilePath	服务器存储路径, 指向 CSV 文件
FileSize	文件大小
UploadTimestamp	上传时间
NumberOfRows	数据行数
NumberOfFeatures	特征数量
Status	状态, 例如: ‘已上传’, ‘预处理中’, ‘可用’
Description	数据集描述, 可选

(3) 模型表: 包含模型 ID、模型名称、模型类型、模型存储路径等字段。主要功能是存储用户训练或导入的机器学习模型信息。表结构如下表 4.3 所示。

表 4.3 模型表主要字段和字段描述

主要字段	描述
ModelID	主键, 模型 ID
UserID	外键, 关联用户表, 记录创建者/导入者
ModelName	模型名称, 用户自定义或系统生成

续表 4.3 模型表主要字段和字段描述

主要字段	描述
ModelType	模型类型, 例如: ‘LightGBM’, ‘XGBoost’, ‘CatBoost’, ‘加权投票融合模型’
FilePath	服务器存储路径, 指向模型文件, 如. pkl 格式
CreationTimestamp	创建/导入时间
PerformanceMetrics	性能指标, JSON 或文本格式, 存储如 Macro-F1, 精确率, 召回率等
SourceDatasetID	外键, 关联数据集表, 记录训练该模型所用的数据集 I
Description	模型描述, 可选

(4) 训练任务表: 包含任务 ID、训练参数、开始/结束时间等字段。主要功能是记录和追踪模型训练过程中的详细信息。表结构如下表 4.4 所示。

表 4.4 训练任务表主要字段和字段描述

主要字段	描述
JobID	主键, 任务 ID
UserID	外键, 关联用户表
ModelID	外键, 关联模型表, 训练完成后生成的模型 ID
DatasetID	外键, 关联数据集表, 用于训练的数据集 ID
Parameters	训练参数, JSON 或文本格式, 例如学习率、树的最大深度等
StartTime	开始时间
EndTime	结束时间
Status	状态, 例如: ‘运行中’, ‘已完成’, ‘失败’
LogFilePath	训练日志文件路径, 可选
ValidationResults	验证集评估结果, 例如 F1 分数

(5) 预测任务表: 包含预测任务 ID、预测结果文件存储路径、开始/结束时间等字段。主要功能是记录用户发起的预测任务详情。表结构如下表 4.5 所示

表 4.5 预测任务表主要字段和字段描述

主要字段	描述
PredictionJobID	主键, 预测任务 ID
UserID	外键, 关联用户表
ModelID	外键, 关联模型表, 用于预测的模型 ID
PredictionJobID	主键, 预测任务 ID
UserID	外键, 关联用户表
ModelID	外键, 关联模型表, 用于预测的模型 ID

续表 4.5 预测任务表主要字段和字段描述

主要字段	描述
DatasetID	外键, 关联数据集表, 用于预测的输入数据集 ID
StartTime	开始时间
EndTime	结束时间
Status	状态, 例如: ‘运行中’, ‘已完成’, ‘失败’
OutputFilePath	预测结果文件存储路径

数据库设计对系统功能的支持:

(1) 数据可视化模块, 主要依赖数据集表获取数据集元信息, 如文件名、大小、特征数等相关信息, 进行展示。

(2) 模型训练模块, 用户上传数据集时, 信息存入数据集表; 训练任务开始时, 相关参数、数据集信息、用户信息存入训练任务表; 训练完成后, 生成的模型信息存入模型表, 并更新训练任务表中的 ModelID 和状态。模型文件和训练日志的路径也在此记录。

(3) 预测模块, 用户上传待预测数据集时, 信息可存入数据集表; 用户选择模型(从模型表加载)进行预测时, 任务信息存入预测任务表; 预测结果文件的路径在预测任务表中记录。

在系统实现层面, 可以结合 Flask 框架和 SQLAlchemy 等 ORM 工具, 方便地进行数据库操作。

4.3 系统实现

本小节将详细阐述系统的具体实现过程。系统遵循 4.1 节的需求分析内容和 4.2 节的系统设计内容, 采用 B/S 架构, 后端基于 Python 的 Flask 框架搭建, 前端则采用 Vue 框架搭建用户交互界面。数据库层面, 如 4.2.2 节所述, 选用 MySQL 关系型数据库, 并通过 SQLAlchemy ORM 工具进行数据操作, 实现对数据的管理。

4.3.1 开发环境与技术选型

后端技术栈:

(1) 编程语言: Python 3.10

(2) Web 后端框架: Flask(用于构建 Web 应用程序的路由、请求处理和 API 接口)

(3) 数据处理与机器学习: Pandas (数据处理), Scikit-learn (机器学习工具库), LightGBM, XGBoost, CatBoost (核心模型库)

(4) 数据库 ORM: SQLAlchemy (用于数据库交互)

(5) 模型持久化: Joblib 库 (用于保存和加载训练好的模型)

前端技术栈:

(1) 前端框架: Vue.js

(2) 组件库: Element

(3) 数据可视化: ECharts (用于在前端动态展示数据分析结果和模型性能指标)

(4) 异步通信: Axios (用于前后端数据交互)

数据库: MySQL 5.7+

开发工具: VS Code、PyCharm、Git (版本控制)

4.3.2 核心模块实现

根据 4.2.1 节的系统总体功能设计, 系统主要包含数据可视化模块、模型训练模块和故障预测模块。

数据可视化模块实现:

(1) 数据上传与解析: 前端提供文件上传接口, 用户可上传 CSV 格式的数据集。后端 Flask 接收到文件后, 使用 Pandas 库进行解析 (如检查文件格式)。数据集元信息 (如文件名、大小、上传时间、特征数量、样本数量) 存储在数据库中的数据集中表中。

(2) 数据统计与可视化: 后端对上传的数据进行统计分析, 如计算数据集总样本数量、缺失样本数量、重复样本数量、类别分布情况、各特征缺失值统计以及数据的二维空间分布图。这些数据通过 Pandas 进行统计计算, 然后以 json 的格式发送给前端。前端利用 ECharts 库, 将这些数据显示为直方图、饼图和散点图, 帮助用户更好地了解数据的特性。

模型训练模块实现:

(1) 训练配置: 用户在前端界面上传自己的训练集和测试集, 并配置模型训练的相关参数。

(2) 数据预处理: 在模型训练前, 后端会自动对上传的数据进行数据预处理流程, 包括数据去重、常数特征移除等操作。

(3) 模型训练与调优：后端 Flask 应用接收到训练请求后，会启动一个训练任务。该任务会根据用户选择或系统默认配置，训练 LightGBM、XGBoost、CatBoost 等基模型。

模型存储与管理：训练完成后，生成的模型文件（joblib 格式）被保存在服务器指定路径，模型元信息（模型 ID、名称、类型、创建时间、性能指标如 Macro-F1、源数据集 ID 等）以及训练任务的元信息（如参数、开始/结束时间、状态）分别存入数据库中的模型表和训练任务表。用户可以下载以及训练好的模型

故障预测模块实现：

(1) 数据上传与模型选择：用户上传待预测的 CSV 格式数据集，可以选择系统内置的训练好的模型或者自己导入模型文件。

(2) 预测执行：后端 Flask 应用接收到预测请求后，加载用户选定的模型文件。对待预测数据执行与训练时一致的预处理操作。然后，使用加载的模型进行预测。

(3) 结果展示与下载：预测结果（每个样本的故障类别）会返回给前端进行展示。用户可以查看预测详情。预测任务的信息（如使用的模型 ID、输入数据集 ID、预测时间、输出文件路径）会存入到数据库中的预测任务表。

4.3.3 前后端交互

系统前后端之间主要通过 RESTful API 进行通信。前端通过异步请求(Axios)将用户操作（如文件上传、表单提交）和数据发送到后端 Flask 定义的 API 端点。后端处理请求后，以 JSON 格式返回数据或状态信息给前端，前端再根据响应动态更新页面内容。

4.3.4 用户界面展示

数据可视化界面：提供数据上传入口，并使用 ECharts 图表展示数据集样本数量、含缺失值样本数量、重复样本数量、类别分布图、各特征缺失值统计、各类别二维空间分布图（如图 4.3 所示）。

模型训练界面：提供数据集上传入口，支持用户上传训练集以及测试集。允许用户选择模型类型以及自定义模型名称。配置完毕后点击确定提交训练任务，训练完成后显示模型训练结果以及各类评价指标图，支持用户下载训练好的模型（如图 4.4）。

预测任务界面：提供需要预测的数据集上传入口，支持用户可以上传预测数据集。上传后可以在线预览数据。提供预测任务配置入口，用户可以选择预测的模型或者上传自己的模型文件。配置完毕后提交预测任务，展示预测的结果并支持下载预测结果，展示各类别预测分布（如图 4.5 所示）。

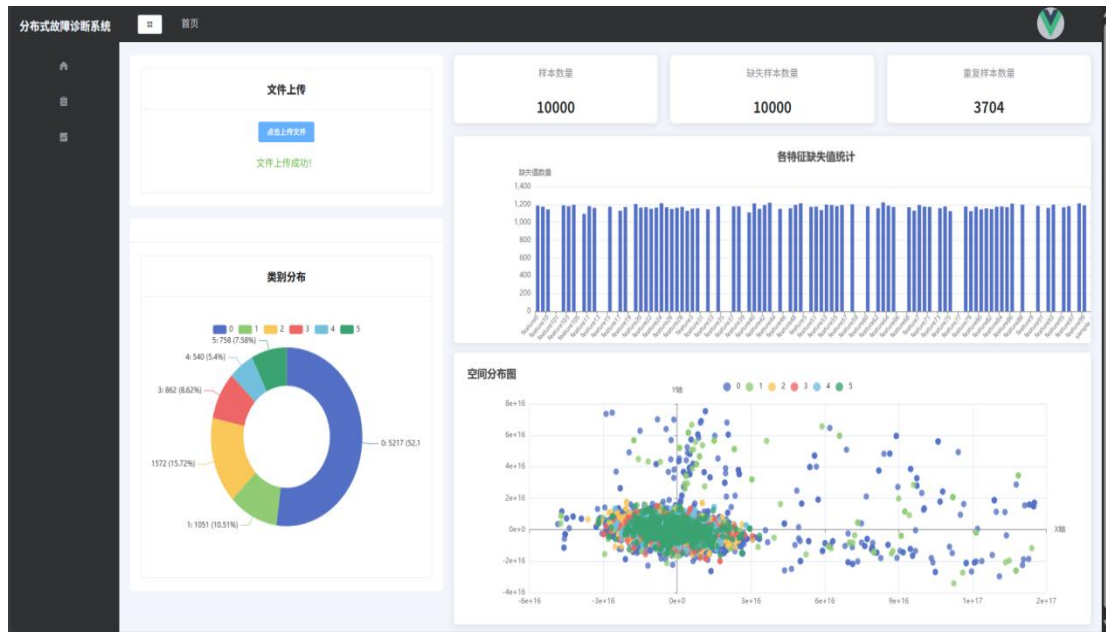


图 4.3 数据可视化界面展示

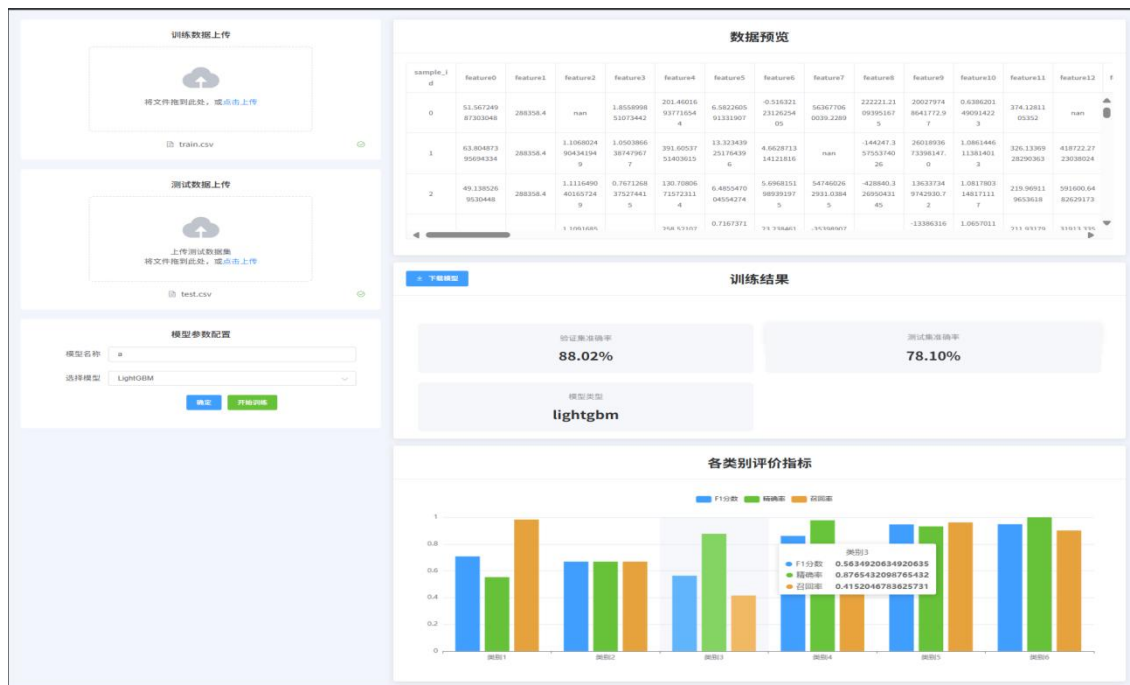


图 4.4 训练任务界面展示

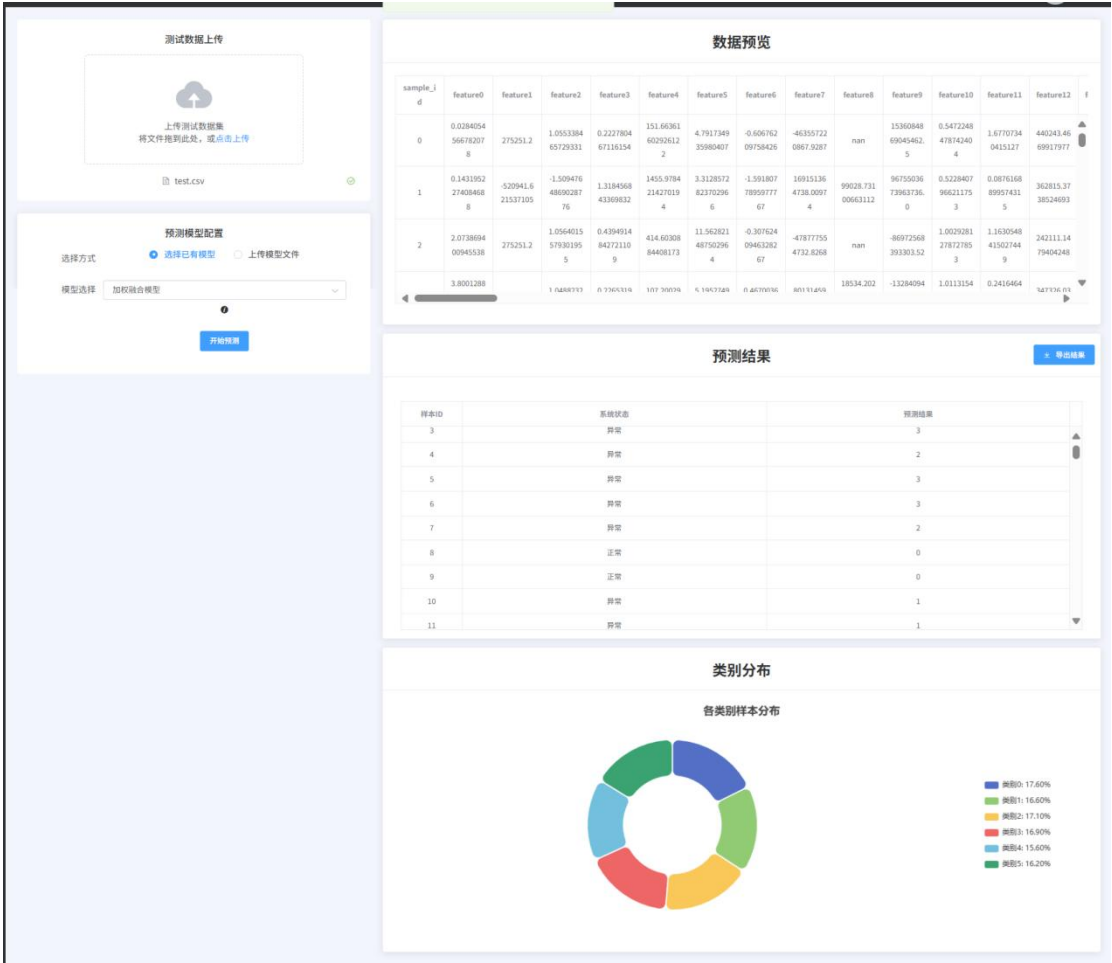


图 4.5 训练任务界面展示

4.4 小结

通过上述实现方案，构建了一个功能完整、流程清晰的基于机器学习的分布式系统故障检测平台。该平台能够支持从数据接入、可视化分析到模型训练、优化及最终的故障预测全过程，为运维工程师提供了有力的辅助工具。

第五章 总结与展望

5.1 总结

文章围绕分布式系统故障检测展开研究，针对传统分布式系统难以维护的问题，设计基于机器学习的分布式系统故障检测系统可以帮助运维人员高效地分析问题，并提升故障诊断的准确性和效率。

研究所用数据集包含 107 个 KPI 特征和 6 类故障标签（含无故障类别），通过数据分析可以发现，该数据集主要存在数据重复、数据缺失、类别分布不平衡和特征分布不平衡四类问题，其中数据缺失以及类别分布最为严重。所有样本均有缺失值，77 个特征存在缺失，平均缺失率 8.44%，且缺失模式为多变量随机缺失，缺失机制为 NMAR。数据类别分布不均匀，其中 0 类别（正常类别）占比超过 50%，

为了解决上述提出的问题，文章提出并实施了一套针对性的数据预处理思路。首先解决数据重复的问题，针对数据集直接采用删除重复值的操作，数据集由原来的 10000 条样本数据下降到 6296 条数据，去重可以避免重复数据在后续训练中给模型带来负面影响。通过对比实验，发现保留缺失值（不进行填充）的效果优于多重插补 MCMC 法，推断缺失值本身可作为故障传播的证据。利用 LightGBM 模型内置的样本权重机制，提升对少数类样本的关注度。移除了 3 个常数特征，并依赖 LightGBM 模型自身的特征选择能力，未进行额外的显式特征选择。

在模型构建、训练以及优化部分，选用 LightGBM 作为核心基准模型。采用遗传算法、贝叶斯优化和网格搜索相结合的混合策略对 LightGBM 的超参数进行调优，有效提升了模型的拟合能力和泛化性能，使 Macro-F1 分数从 0.75 提升至 0.849。为进一步提升性能，引入 XGBoost 和 CatBoost 作为额外的基模型，并通过加权投票的融合策略将三个模型进行集成。

通过上述数据预处理、模型选择、超参数优化和模型融合策略，最终构建的集成模型在验证集上的 Macro-F1 分数达到了 0.855，相较于初始的 LightGBM 模型的 0.75，有了显著的性能提升，证明了本研究提出方法的有效性。

5.2 未来展望

基于本研究的基础，未来可以从以下几个方面进行更深入地探索：

- （1）使用更复杂的融合策略：尝试使用 Stacking 等更复杂的模型融合技术，

以期进一步挖掘模型集成的潜力。

（2）深度学习模型的应用：探索将深度学习模型（使用 TransFormers 或者 LSTM 等深度学习模型）应用于分布式系统故障检测任务，特别是在处理时序 KPI 数据方面可能具有优势。

（3）特征工程与可解释性：进一步深入分析特征与故障类别之间的关系，探索更优秀的特征工程方法，并加强模型的可解释性研究，以便更好地理解故障发生原理和机制。

（4）实时故障检测系统优化：结合本研究的算法成果，进一步优化子系统，使系统具备更加良好的功能，为用户带来更良好的体验。

参考文献

- [1] Maciel A M A .Fault Detection and Diagnosis in Industry 4.0: A Review on Challenges and Opportunities[J].Sensors, 2024, 25.DOI:10.3390/s25010060.
- [2] 汪俊杰,游科友,彭冬亮,等.基于阈值判断的GA-BP的磁悬浮列车故障检测方法:201911038991[P][2025-05-08].
- [3] 胡静.基于多元统计分析的故障诊断与质量监测研究[D].浙江大学,2015.
- [4] 裴丹,李明杰,卢香琳.一种基于多模态数据依赖关系的故障诊断方法和装置:CN202210676227.2[P].CN202210676227.2[2025-05-08].
- [5] 刘梅,周洪萍,丁文怡.基于机器学习的分布式系统故障诊断实现方法[J].无线互联科技, 2024, 21(20):113-117.
- [6] 胡旭峰.基于无监督学习的故障诊断算法研究[D].山东大学, 2022.DOI:10.27272/d.cnki.gshdu.2022.000045.
- [7] 刘旭媛.基于半监督学习的空调设备故障检测和诊断研究[D].深圳大学, 2022.DOI:10.27321/d.cnki.gszdu.2022.000604.
- [8] 张晋恺,马洁.基于深度学习的滚动轴承故障预测[J].现代电子技术, 2024, 47(24):120-130.DOI:10.16652/j.issn.1004-373x.2024.24.019.
- [9] 雷亚国,贾峰,孔德同,等.大数据下机械智能故障诊断的机遇与挑战[J].机械工程学报, 2018(5):11.DOI:10.3901/JME.2018.05.094.
- [10] Rubin, Donald, B. INFERENCE AND MISSING DATA[J].ETS Research Bulletin Series, 1975.DOI:10.1002/j.2333-8504.1975.tb01053.x.
- [11] 邓建新,单路宝,贺德强,等.缺失数据的处理方法及其发展趋势[J].统计与决策, 2019(23):7.DOI:10.13546/j.cnki.tjyjc.2019.23.005.
- [12] 孙婕,金勇进,戴明锋.关于数据缺失机制的检验方法探讨[J].数学的实践与认识, 2013, 43(12):166-173.
- [13] 邱建青,杜春霖,周婷,等.多变量数据缺失机制的识别方法[J].中国卫生统计, 2017, 34(06):1002-1005.
- [14] 杨浩.针对非平衡数据问题的分类方法研究[D].西南交通大学, 2020.DOI:10.27414/d.cnki.gxnju.2020.001872.
- [15] 张桥,李宁,张秋菊,等.任意缺失模式缺失数据不同填补方法效果比较[J].中国卫生统计, 2013, 30(5):3.DOI:CNKI:SUN:ZGWT.0.2013-05-019.
- [16] Cai J ,Luo J ,Wang S , et al.Feature selection in machine learning: A new perspective[J].Neurocomputing, 2018, 30070-79.
- [17] Meng Q .LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]//Neural

- Information Processing Systems. Curran Associates Inc. 2017.
- [18] 罗山贵, 赵玉龙, 肖红林, 陈伟华, 贺戈, 张烈辉, 杜诚. 基于混合优化算法和深度神经网络模型结合的致密砂岩气藏裂缝参数优化[J]. 天然气工业, 2024, 44(9): 140-151
- [19] 李林玉. 基于机器学习的非标车辆价格预测研究[D]. 南京理工大学, 2022. DOI:10.27241/d.cnki.gnjgu.2022.001431.
- [20] Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System. [J]. CoRR, 2016, abs/1603.02754
- [21] Dorogush A V, Ershov V, Gulin A. CatBoost: gradient boosting with categorical features support[J]. 2018. DOI:10.48550/arXiv.1810.11363.